

**BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ**



**CHUYÊN ĐỀ CƠ SỞ
KHOA: CÔNG NGHỆ THÔNG TIN**

**XÂY DỰNG HỆ THỐNG HONEYPOT TÍCH HỢP MACHINE
LEARNING NHẪM THU THẬP VÀ PHÂN TÍCH TẤN CÔNG WEB**

Sinh viên thực hiện:

Nguyễn Việt Toàn AT200158

Người hướng dẫn:

TS. Lê Đức Thuận

Khoa công nghệ thông tin – Học viện Kỹ thuật mật mã

Hà Nội – 2026

MỤC LỤC

MỤC LỤC	i
DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ	iii
DANH MỤC HÌNH ẢNH.....	iv
DANH MỤC BẢNG BIỂU	v
LỜI NÓI ĐẦU	1
CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI	3
1.1. Khảo sát hệ thống honeypot hiện nay	3
1.2. Sơ lược về hệ thống honeypot.....	5
1.2.1. Khái niệm.....	5
1.2.2. Phân loại.....	6
1.3. Tìm hiểu kỹ thuật môi nhử.....	9
1.3.1. Các kỹ thuật môi nhử phổ biến.....	9
1.3.2. Cơ chế đánh lừa và tương tác	9
1.3.3. Cơ chế thu thập và trích xuất dữ liệu.....	10
1.4. Tìm hiểu về Krawl	10
1.4.1. Giới thiệu tổng thể	10
1.4.2. Nguyên lý hoạt động.....	11
1.4.3. Ưu điểm và hạn chế	12
1.5. Tổng quan học máy	13
1.5.1. Học máy là gì?	13
1.5.2. Các phương pháp học máy phổ biến	13
1.5.3. Ưu và nhược điểm của Machine Learning	16
1.5.4. Ứng dụng thực tế	17
1.6. Tổng kết chương 1	18
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG HONEYPOT TÍCH HỢP MACHINE LEARNING.....	19
2.1. Phân tích yêu cầu và kiến trúc của hệ thống.	19
2.1.1. Mục tiêu cụ thể	19
2.1.2. Các chức năng chính.....	19
2.1.3. Mô hình kiến trúc.....	21
2.2. Triển khai hệ thống	23
2.2.1. Kiến trúc tổng thể của hệ thống.....	23
2.2.2. Cấu hình môi trường internet.....	26

2.2.3. Xây dựng và cấu hình các kịch bản môi thử.....	27
2.3. Thu thập và xử lý dữ liệu	28
2.3.1. Thu thập dữ liệu.....	28
2.3.2. Tiền xử lý dữ liệu.....	29
2.4. Xây dựng và huấn luyện mô hình machine learning	31
2.4.1. Bài toán đưa ra.....	31
2.4.2. Lựa chọn thuật toán	32
2.4.3. Huấn luyện mô hình.....	32
2.4.4. Đánh giá kết quả	34
2.5. Tích hợp mô hình vào hệ thống phân tích	36
2.6. Tổng kết chương 2	37
CHƯƠNG 3 TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	38
3.1. Thực nghiệm	38
3.1.1. Môi trường triển khai.....	38
3.1.2. Các kịch bản thực nghiệm	38
3.2. Kết quả thu được	39
3.2.1. Đánh giá năng lực thu thập của Krawl Honeypot	39
3.2.2. Hiệu năng phân loại của XGBoost	41
3.3. Tổng kết chương 3	46
3.3.1. Các điểm mạnh đạt được	46
3.3.2. Những hạn chế tồn tại.....	46
3.3.3. Hướng phát triển tiếp theo	47
KẾT LUẬN	48
TÀI LIỆU THAM KHẢO	49
PHỤ LỤC	50

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Ký hiệu	Nghĩa tiếng Anh	Nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CMDi	Command Injection	Lỗi hỏng chèn lệnh hệ thống
ELK	Elastic Stack	Nền tảng phân tích và quản lý log tập trung
IDS/IPS	Intrusion Detection System / Intrusion Prevention System	Hệ thống phát hiện / ngăn ngừa xâm nhập
LFI/RFI	Local / Remote File Inclusion	Lỗi hỏng chèn tệp tin cục bộ / từ xa
ML	Machine Learning	Học máy
NTP	Network Time Protocol	Giao thức đồng bộ thời gian mạng.
RDP	Remote Desktop Protocol	Giao thức máy tính từ xa
SMB	Server Message Block	Giao thức mạng dùng để chia sẻ tệp tin và thiết bị trong một mạng
SOC	Security Operations Center	Trung tâm Giám sát An toàn Thông tin
SQLi	SQL Injection	Lỗi hỏng chèn mã SQL
SSH	Secure Shell	Giao thức mạng mã hóa
TF-IDF	Term Frequency - Inverse Document Frequency	Tần suất xuất hiện từ - Nghịch đảo tần suất văn bản
URI/URL	Uniform Resource Identifier / Locator	Định danh / Định vị tài nguyên thống nhất
XSS	Cross-Site Scripting	Lỗi hỏng chèn mã kịch bản chéo trang
Zero-day	Zero-day vulnerability	Lỗi hỏng bảo mật chưa được biết đến hoặc chưa có bản vá

DANH MỤC HÌNH ẢNH

Hình 1.1 Honeypot là gì?	5
Hình 1.2 Phân loại Honeypot	6
Hình 1.3 Krawl honeypot framework	11
Hình 1.4 Một số chức năng của Krawl.....	12
Hình 1.5 Học máy có giám sát	14
Hình 1.6 Học máy không giám sát.....	15
Hình 1.7 Ứng dụng của ML	18
Hình 2.1 Mô hình kiến trúc hệ thống	21
Hình 2.2 Endpoint giả mạo	22
Hình 2.3 Cấu hình Nginx đọc request.....	25
Hình 2.4 Số lượng của từng nhãn thu thập được	28
Hình 2.5 Đặc trưng sau chuẩn hóa	30
Hình 2.6 Quá trình huấn luyện mô hình ML.....	33
Hình 2.7 Kết quả đánh giá mô hình Decision Tree.....	34
Hình 2.8 Kết quả đánh giá mô hình Random Forest.....	35
Hình 2.9 Kết quả đánh giá mô hình XGBoost	35
Hình 3.1 Giao diện web trả về phản hồi 200 OK cho một request độc hại	40
Hình 3.2 Các request bắt được từ các cuộc tấn công	40
Hình 3.3 Kết quả hiển thị trên trình duyệt khi truy cập đường dẫn /login.....	41
Hình 3.4 Kết quả hiển thị trên trình duyệt khi truy cập đường dẫn /search.....	42
Hình 3.5 Hệ thống hiển thị phân loại luồng dữ liệu mang nhãn Normal.....	42
Hình 3.6 Giao diện trình duyệt khi nhập payload Path Traversal.....	43
Hình 3.7 Terminal hiển thị nhận diện và cảnh báo nhãn File_Attack	43
Hình 3.8 Thao tác gửi payload SQL Injection vào ô tìm kiếm.....	44
Hình 3.9 Thao tác gửi payload SQL injection vào form đăng nhập	44
Hình 3.10 Terminal Python phân loại và cảnh báo nhãn Injection.....	44
Hình 3.11 Thao tác gửi request chứa mã độc JavaScript (XSS) vào ô tìm kiếm....	45
Hình 3.12 Thao tác gửi request chứa mã độc XSS vào form đăng nhập	45
Hình 3.13 Terminal Python phân loại và cảnh báo nhãn XSS.....	46

DANH MỤC BẢNG BIỂU

Bảng 1: So sánh một số honeypot	4
Bảng 2: So sánh giữa 3 mô hình machine learning.....	36
Bảng 3 Thông số môi trường thực nghiệm	38

LỜI NÓI ĐẦU

Trong kỷ nguyên chuyển đổi số, các ứng dụng web đã trở thành nền tảng cốt lõi duy trì mọi hoạt động giao thương, quản lý và vận hành của hầu hết các tổ chức. Sự phụ thuộc sâu sắc này đồng thời biến các bề mặt ứng dụng thành mục tiêu tấn công hàng đầu của tội phạm mạng. Các chiến dịch khai thác lỗ hổng bảo mật nghiêm trọng như chèn mã SQL, chèn lệnh hệ thống, tấn công kịch bản chéo trang hay duyệt thư mục trái phép đang diễn ra với tần suất ngày càng cao, đi kèm với những thủ đoạn lẩn tránh vô cùng tinh vi. Để đối phó với làn sóng tấn công này, các tổ chức thường trang bị các lớp phòng thủ thụ động như tường lửa ứng dụng web hoặc hệ thống phát hiện xâm nhập. Mặc dù đóng vai trò nền tảng, các giải pháp này đang dần bộc lộ những hạn chế nhất định. Do phần lớn hoạt động dựa trên cơ chế nhận diện chữ ký tĩnh, chúng gặp rất nhiều khó khăn trước các biến thể mã độc mới hoặc các lỗ hổng chưa từng được công bố. Hậu quả là các hệ thống phòng thủ truyền thống thường xuyên sinh ra lượng lớn cảnh báo giả, gây ra áp lực quá tải cho đội ngũ rà soát và dễ dàng bỏ lọt những mối đe dọa có chủ đích.

Từ thực trạng đó, cùng với nền tảng kiến thức tích lũy sau ba năm theo học chuyên ngành An toàn thông tin, em nhận thấy sự chuyển dịch từ phòng thủ thụ động sang phòng thủ chủ động là một xu thế tất yếu. Trong mạng lưới phòng thủ hiện đại, công nghệ mồi nhử Honeypot nổi lên như một giải pháp thu thập thông tin tình báo chiến lược. Bằng cách thiết lập các môi trường web giả lập để giăng bẫy tin tặc, hệ thống có thể an toàn ghi nhận lại toàn bộ công cụ, thủ đoạn và mã khai thác mà không gây rủi ro cho hạ tầng thực tế. Tuy nhiên, thách thức lớn nhất khi vận hành hệ thống này là khối lượng nhật ký sự kiện thu về từ máy chủ thường vô cùng đồ sộ. Việc bóc tách và rà soát thủ công khối lượng dữ liệu khổng lồ này là bất khả thi và kém hiệu quả. Nhu cầu cấp thiết đặt ra là phải có một cơ chế phân tích và đối soát tự động. Đây chính là động lực cốt lõi thúc đẩy em quyết định lựa chọn đề tài: "Xây dựng hệ thống Honeypot tích hợp Machine Learning nhằm thu thập và phân tích tấn công Web" cho học phần Chuyên đề cơ sở.

Dù được giới hạn ở quy mô một chuyên đề môn học, đề tài vẫn hướng tới việc giải quyết bài toán thực tiễn với ba mục tiêu rõ ràng. Về mặt hạ tầng an toàn thông tin, đề tài tập trung nghiên cứu và triển khai thành công một hệ thống máy chủ mồi nhử ứng dụng nền tảng Krawl trên môi trường Windows, nhằm tạo ra một lưới bẫy đủ độ chân thực để thu hút, đánh lừa và ghi nhận trọn vẹn các luồng dữ liệu tấn công web từ bên ngoài. Về mặt xử lý dữ liệu và trí tuệ nhân tạo, nghiên cứu ứng dụng các thuật toán Học máy cơ bản bằng ngôn ngữ Python để thiết lập đường ống xử lý dữ liệu tự động. Hệ thống sẽ tiến hành làm sạch, chuẩn hóa và tự động phân loại, gom cụm các chuỗi hành vi tấn công từ kho dữ liệu nhật ký thô thu thập được. Thông qua quá trình thực hiện đề tài, em kỳ vọng sẽ củng cố vững chắc kiến thức về lỗ hổng

bảo mật web, rèn luyện kỹ năng lập trình xử lý dữ liệu lớn, đồng thời tiếp cận sâu hơn với quy trình vận hành và giám sát an toàn thông tin trong môi trường thực chiến.

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI

1.1. Khảo sát hệ thống honeypot hiện nay

Trong bối cảnh các cuộc tấn công mạng ngày càng tinh vi và tự động hóa cao, việc phòng thủ thụ động không còn đủ để bảo vệ hệ thống. Các tổ chức hiện nay đang có xu hướng chuyển dịch sang các biện pháp phòng thủ chủ động, trong đó Honeypot đóng vai trò mũi nhọn.

Đặc biệt, trong quy trình vận hành của các Trung tâm Giám sát An toàn Thông tin (SOC), honeypot đang được nâng cấp thành công nghệ đánh lừa, rải rác các "mồi nhử" khắp mạng nội bộ để phát hiện sớm các hành vi của kẻ tấn công.

Cộng đồng mã nguồn mở và các tổ chức an ninh mạng hiện nay cung cấp nhiều giải pháp honeypot chuyên dụng cho từng loại mục tiêu tấn công khác nhau:

- Nền tảng tích hợp T-Pot: Đây là một trong những nền tảng phổ biến nhất hiện nay do Deutsche Telekom phát triển. T-Pot đóng gói hàng chục honeypot chuyên dụng (như Cowrie, Dionaea, Mailoney, Honeytrap...) vào các container Docker trên nền hệ điều hành Linux. Điểm mạnh của T-Pot là tích hợp sẵn hệ thống Elastic Stack (ELK) giúp trực quan hóa dữ liệu log thu thập được (bản đồ IP tấn công, danh sách mật khẩu phổ biến, loại mã độc) thông qua các dashboard trực quan theo thời gian thực.[1]
- Cowrie (Honeypot tương tác trung bình): Chuyên dụng cho việc giả lập dịch vụ SSH và Telnet. Cowrie có khả năng ghi lại toàn bộ phiên tương tác của kẻ tấn công, lưu lại các câu lệnh bash/shell được gõ, các tên đăng nhập/mật khẩu được dùng để brute-force, đồng thời bắt giữ và lưu lại các tệp tin mã độc (malware) mà tin tặc cố tình tải lên hệ thống bẫy.[2]
- SNARE / Glastopf: Thay vì giả lập một website tĩnh, các công cụ này có khả năng phản hồi động lại các truy vấn từ kẻ tấn công. Chúng giả lập các lỗ hổng ứng dụng Web kinh điển như SQL Injection, Local/Remote File Inclusion (LFI/RFI) nhằm thu thập các chuỗi payload tấn công và nhận diện các công cụ rà quét tự động.[3]
- Krawl là một máy chủ mồi nhử web mã nguồn mở, nhẹ và tối ưu cho nền tảng đám mây. Nó hoạt động bằng cách tạo ra các ứng dụng web giả mạo chứa các lỗ hổng cơ bản (như trang đăng nhập giả, thư mục chứa thông tin xác thực) để đánh lừa và theo dõi tin tặc cũng như các công cụ quét tự động.[6]

Bảng 1: So sánh một số honeypot

	T-Pot	Cowrie	SNARE / Glastopf	Krawl
Bản chất hệ thống	Nền tảng tích hợp đa tầng (Multi-Honeypot Platform).	Honeypot chuyên dụng tầng hệ thống.	Honeypot chuyên dụng tầng ứng dụng Web.	Hệ thống đánh lừa tầng Web (Web Deception Server).
Mức độ tương tác	Đa mức độ (Tùy thuộc vào honeypot con bên trong).	Tương tác trung bình (Medium-Interaction).	Tương tác thấp đến trung bình.	Tương tác trung bình (Tập trung mô phỏng cấu trúc).
Dịch vụ mục tiêu	Đa nền tảng (HTTP, SSH, SMB, FTP, v.v.).	Giao thức SSH (Port 22) và Telnet (Port 23).	Giao thức HTTP/HTTPS (Nhắm vào lỗ hổng web).	Giao thức HTTP/HTTPS (Nhắm vào Botnet, Scanners, API).
Đặc điểm nổi bật	Đóng gói hàng chục honeypot vào Docker. Tích hợp sẵn Elastic Stack giúp trực quan hóa log theo thời gian thực.	Giả lập môi trường shell. Ghi lại toàn bộ phiên tương tác, câu lệnh bash/shell và bắt giữ các tệp tin mã độc.	Phản hồi động các truy vấn thay vì web tĩnh. Giả lập các lỗ hổng kinh điển (SQLi, LFI/RFI) để bắt payload từ các công cụ rà quét.	Tự động sinh ra cấu trúc web giả chân thực (admin panel, thư mục ẩn, API ảo). Chuyên bắt các bộ thu thập dữ liệu và dò quét.
Ứng dụng	Phù hợp triển khai cho SOC quy mô lớn, cần thu thập tình báo (Threat Intelligence) từ nhiều bề mặt tấn công.	Lý tưởng để phân tích chiến dịch Brute-force mật khẩu máy chủ và nghiên cứu hành vi leo thang đặc quyền.	Chuyên dùng để đánh lừa và nhận diện kỹ thuật của các trình rà quét lỗ hổng tự động.	Rất phù hợp để kết hợp làm đường ống trích xuất dữ liệu Request sạch, cung cấp đầu vào chất lượng cho mô hình Machine Learning.

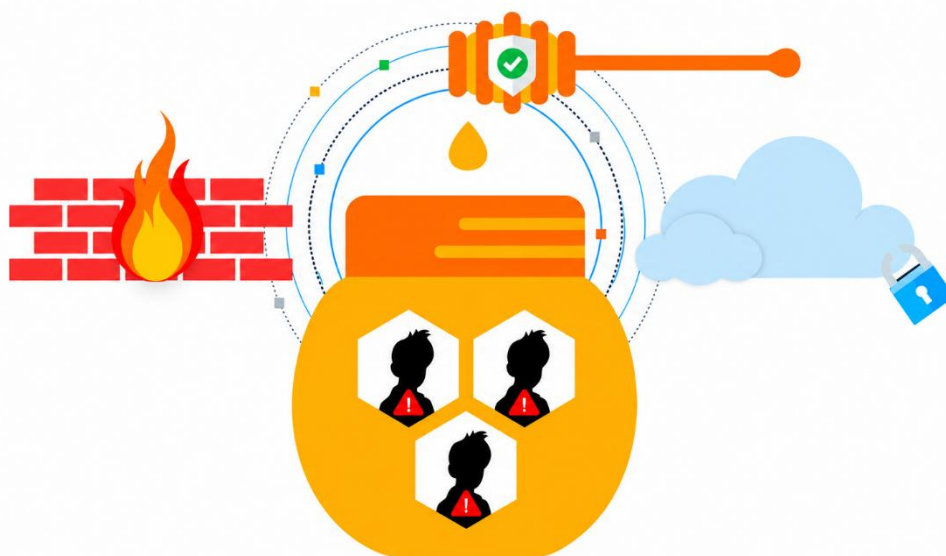
1.2. Sơ lược về hệ thống honeypot

1.2.1. Khái niệm

Honeypot là một phần mềm hoặc thiết lập điều khiển có mục tiêu để thu hút các hành vi không mong muốn như tấn công mạng, thâm nhập trái phép hoặc lừa đảo. Honeypot thường được sử dụng để giám sát và thu thập thông tin về các kỹ thuật tấn công, nhận diện các mối đe dọa tiềm ẩn và bảo vệ các hệ thống mạng khỏi các mối đe dọa này. (Hình 1.1)

Mục tiêu chính của honeypot là thu thập dữ liệu về kẻ tấn công và phương pháp tấn công của họ, từ đó cung cấp thông tin quý giá để cải thiện biện pháp bảo mật. Điều này không chỉ nâng cao nhận thức về an ninh mạng mà còn hỗ trợ xây dựng các chiến lược phòng thủ hiệu quả hơn.

Khái niệm honeypot không mới; nó đã xuất hiện từ những năm 1990. Ban đầu, honeypot được phát triển như một công cụ nghiên cứu nhằm giúp các nhà nghiên cứu an ninh mạng hiểu sâu hơn về các cuộc tấn công mạng.



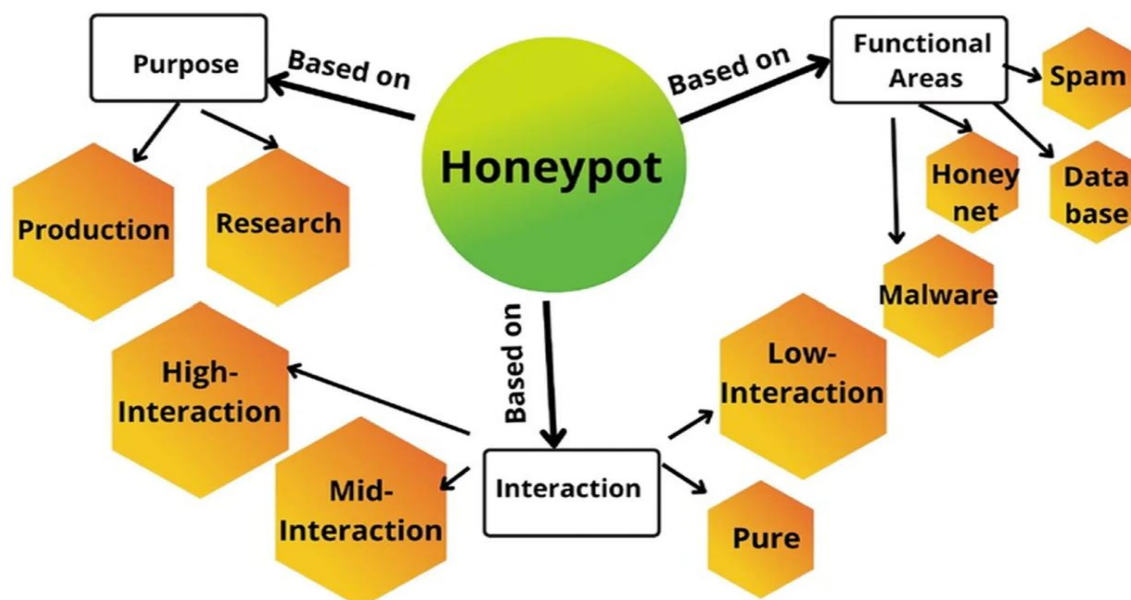
Hình 1.1 Honeypot là gì?

Khi internet phổ biến và các ứng dụng, dịch vụ trực tuyến phát triển, số lượng cuộc tấn công mạng tăng nhanh. Việc theo dõi và phân tích các cuộc tấn công thực tế trên các hệ thống quan trọng thường rất khó khăn. Honeypot được phát triển như một giải pháp để tạo môi trường giả lập, thu hút kẻ tấn công.

Theo thời gian, honeypot đã phát triển từ các mô hình đơn giản đến các hệ thống phức tạp hơn, có khả năng tự động hóa việc thu thập và phân tích dữ liệu. Ngày nay, honeypot không chỉ được dùng cho nghiên cứu mà còn được áp dụng rộng rãi trong các tổ chức để tăng cường khả năng phòng thủ.

1.2.2. Phân loại

Để triển khai và vận hành hiệu quả trong môi trường thực tế, hệ thống honeypot thường được phân loại dựa trên ba tiêu chí cơ bản: mục đích sử dụng của tổ chức, mức độ tương tác và khu vực chức năng. Sự phân loại này giúp các nhà quản trị an toàn thông tin lựa chọn được mô hình phù hợp nhất với tài nguyên và chiến lược phòng thủ của mình. (Hình 1.2)



Hình 1.2 Phân loại Honeypot

a) Phân loại dựa theo mục đích sử dụng

Tùy thuộc vào chiến lược triển khai của tổ chức, honeypot được chia thành hai nhóm chính với những mục tiêu hoàn toàn khác biệt:

- Honeypot sản xuất: Đây là loại bẫy được thiết kế để triển khai trực tiếp bên trong mạng nội bộ của các doanh nghiệp hoặc tổ chức. Đóng vai trò như một thành phần trong kiến trúc phòng thủ chiều sâu, honeypot sản xuất thường được ngụy trang giống hệt các máy chủ nghiệp vụ thực tế. Mục tiêu cốt lõi của chúng không phải là để nghiên cứu chuyên sâu về mã độc, mà là đánh lạc hướng tin tặc ra khỏi các tài sản quan trọng. Khi kẻ tấn công xâm nhập vào mạng nội bộ và thực hiện các hành vi rà quét di chuyển ngang, chúng sẽ vướng vào các hệ thống mồi nhử này. Nhờ đó, honeypot sản xuất giúp kéo dài thời gian tấn công của tin tặc và tạo ra các cảnh báo có độ tin cậy cao, hỗ trợ đội ngũ giám sát an toàn thông tin (SOC) phản ứng và ngăn chặn sự cố kịp thời.
- Honeypot nghiên cứu: Khác với honeypot sản xuất, honeypot nghiên cứu thường được các chuyên gia, nhà nghiên cứu bảo mật hoặc các tổ chức an ninh mạng đặt trực tiếp trên môi trường Internet công cộng. Chức năng

chính của hệ thống này là trở thành một trạm thu thập tình báo mỗi đe dọa. Bằng cách chủ động hứng chịu các đợt tấn công từ bên ngoài, honeypot nghiên cứu cho phép thu thập lượng lớn dữ liệu về các chiến dịch rà quét tự động, phân tích các mẫu phần mềm độc hại mới chưa từng xuất hiện, và đi sâu vào việc nghiên cứu phương thức cũng như thủ đoạn hoạt động của các nhóm tội phạm mạng. Dữ liệu thu được từ đây thường được dùng để cập nhật luật cho hệ thống tường lửa hoặc huấn luyện các mô hình phát hiện tấn công.

b) Phân loại dựa theo mức độ tương tác

Mức độ tương tác xác định giới hạn mà hệ thống honeypot cho phép kẻ tấn công can thiệp và thực thi lệnh. Yếu tố này quyết định trực tiếp đến rủi ro bảo mật và chất lượng dữ liệu thu thập được.

- Honeypot tương tác thấp: Đây là loại bẫy đơn giản nhất, chỉ tập trung vào việc mô phỏng các cổng dịch vụ mạng cơ bản hoặc phản hồi lại các gói tin TCP/IP chuẩn. Kẻ tấn công không thể tương tác trực tiếp với hệ điều hành ở tầng sâu. Ưu điểm lớn nhất của mô hình này là cực kỳ dễ triển khai, tiêu tốn rất ít tài nguyên phần cứng và rủi ro bị chiếm quyền kiểm soát gần như bằng không. Tuy nhiên, do tính chất mô phỏng đơn giản, những kẻ tấn công có kinh nghiệm sẽ dễ dàng nhận ra đây là hệ thống giả mạo. Dữ liệu thu thập được chủ yếu chỉ dừng lại ở các thông tin quét cổng mạng cơ bản.
- Honeypot tương tác trung bình: Mô hình này mang lại sự cân bằng hoàn hảo giữa rủi ro bảo mật và lượng thông tin thu thập được. Thay vì chỉ mở cổng dịch vụ, honeypot tương tác trung bình giả lập chi tiết hơn về mặt hệ điều hành và các dịch vụ ứng dụng. Nó có khả năng đánh lừa tin tặc tiến hành các bước tấn công sâu hơn, chẳng hạn như cho phép nhập mật khẩu dò đoán, thực thi một số tập lệnh hệ thống cơ bản hoặc tải xuống các tệp tin chứa mã độc. Mặc dù kẻ tấn công có cảm giác như đang kiểm soát được hệ thống, nhưng thực chất chúng chỉ đang tương tác với một môi trường mô phỏng ảo, hoàn toàn không có một hệ điều hành lõi thực sự ở phía dưới để bị lợi dụng.
- Honeypot tương tác cao: Đúng như tên gọi, đây là một hệ thống máy chủ thật được cung cấp đầy đủ hệ điều hành, phần mềm và các lỗ hổng bảo mật cố ý. Hệ thống này cho phép kẻ tấn công tương tác và kiểm soát toàn bộ máy chủ sau khi khai thác thành công. Nhờ đó, các nhà nghiên cứu có thể thu thập được dữ liệu cực kỳ chi tiết về toàn bộ vòng đời của một cuộc tấn công, bao gồm cả những hành vi leo thang đặc quyền hay cài đặt Rootkit phức tạp. Tuy nhiên, nhược điểm lớn nhất là việc triển khai đòi hỏi tài nguyên khổng lồ và đi kèm với rủi ro bảo mật cực kỳ nghiêm trọng. Nếu

không được cấu hình cách ly mạng một cách chặt chẽ, kẻ tấn công hoàn toàn có thể sử dụng chính máy chủ honeypot này làm bàn đạp để phát động tấn công sang các hệ thống khác.

c) Phân loại dựa theo khu vực chức năng

Bên cạnh mục đích và mức độ tương tác, hệ thống honeypot còn được phân loại dựa trên các dịch vụ cụ thể mà chúng mô phỏng hoặc loại hình tấn công mà chúng nhắm đến. Cách phân loại này giúp làm rõ vai trò chuyên biệt của từng công cụ trong mạng lưới phòng thủ:

- Honeypot thu thập mã độc: Đây là các hệ thống được thiết kế chuyên biệt để bắt giữ và thu thập các mẫu phần mềm độc hại lây lan tự động qua mạng, chẳng hạn như Ransomware hay Worms. Chúng thường giả lập các lỗ hổng trên các giao thức chia sẻ file hoặc dịch vụ mạng phổ biến (như SMB, FTP). Khi các dòng mã độc quét qua và nỗ lực lây nhiễm, honeypot sẽ lừa chúng thả lại đoạn mã để phục vụ cho quá trình dịch ngược và phân tích hành vi.
- Honeypot chống thư rác: Mục tiêu của loại bẫy này là thu hút các luồng thư rác và email lừa đảo từ những kẻ phát tán. Bằng cách cố tình để lộ các địa chỉ email ảo hoặc giả lập một máy chủ cấu hình mở, hệ thống sẽ ghi nhận lại các địa chỉ IP độc hại và các mẫu nội dung email lừa đảo. Dữ liệu này sau đó được dùng để cập nhật luật trực tiếp cho các bộ lọc chống thư rác của tổ chức.
- Honeypot cơ sở dữ liệu: Loại bẫy này đặc biệt quan trọng trong việc bảo vệ dữ liệu nhạy cảm. Chúng giả lập các hệ quản trị cơ sở dữ liệu phổ biến (như MySQL, PostgreSQL, Redis) và chứa các "dữ liệu môi" trông giống như thông tin thật (ví dụ: danh sách thẻ tín dụng ảo, mật khẩu giả). Mục đích là để thu hút những kẻ tấn công đang tìm cách đánh cắp dữ liệu hoặc rà quét các lỗ hổng như SQL Injection.
- Mạng bẫy chim mồi: Thay vì chỉ là một máy chủ hay dịch vụ đơn lẻ, Honeynet là một mạng lưới phức tạp bao gồm nhiều hệ thống honeypot kết nối với nhau. Nó giả lập một hệ thống mạng doanh nghiệp hoàn chỉnh với đầy đủ các thành phần như máy chủ web, cơ sở dữ liệu, tường lửa và thiết bị định tuyến ảo. Honeynet giúp các nhà nghiên cứu theo dõi toàn cảnh quá trình di chuyển ngang của tin tặc sau khi chúng xâm nhập thành công vào một điểm trong mạng.

1.3. Tìm hiểu kỹ thuật mồi nhử

1.3.1. Các kỹ thuật mồi nhử phổ biến

Để thu hút tin tặc và các công cụ rà quét tự động, hệ thống honeypot áp dụng các kỹ thuật giăng bẫy đa dạng nhằm tạo ra một mục tiêu trông có vẻ hấp dẫn và chứa nhiều tài nguyên giá trị. Các kỹ thuật nổi bật bao gồm:

- Mở các cổng dịch vụ nhạy cảm: Hệ thống cố tình phơi bày các dịch vụ quản trị từ xa thường xuyên là mục tiêu của các cuộc tấn công dò đoán mật khẩu (brute-force) như SSH (Cổng 22), RDP (Cổng 3389), Telnet (Cổng 23), hoặc các hệ quản trị cơ sở dữ liệu phổ biến. Sự xuất hiện của các cổng mở này sẽ nhanh chóng lọt vào tầm ngắm của các công cụ rà quét mạng.
- Giả lập lỗ hổng ứng dụng Web: Đưa ra các endpoint ảo chứa các lỗ hổng kinh điển. Hệ thống sẽ cố tình để lộ các form đăng nhập thiếu cơ chế xác thực, các thư mục quản trị ẩn (như /admin, /backup), hoặc các tham số URL có khả năng bị chèn mã (SQLi, Command Injection, Path Traversal). Ngay khi phát hiện các điểm yếu này, tin tặc sẽ lập tức bị thu hút và gửi payload khai thác.
- Sử dụng dữ liệu giả mạo: Đây là kỹ thuật rải các "mảnh mồi" vô giá trị nhưng được ngụy trang thành tài sản cực kỳ quan trọng đối với tin tặc. Đó có thể là một tệp cấu hình chứa mật khẩu quản trị (config.php, .env), một cơ sở dữ liệu chứa thông tin thẻ tín dụng ảo, hoặc các khóa API giả mạo. Vì hệ thống thực không bao giờ sử dụng đến các dữ liệu này, bất kỳ nỗ lực truy cập nào nhắm vào Honeytokens đều ngay lập tức bị hệ thống đánh dấu là hành vi độc hại.

1.3.2. Cơ chế đánh lừa và tương tác

Sau khi các kỹ thuật mồi nhử thu hút được sự chú ý của kẻ tấn công, hệ thống cần vận hành các cơ chế tương tác khéo léo để lừa tin tặc bộc lộ toàn bộ công cụ và thủ đoạn mà không nhận ra mình đang ở trong bẫy:

- Phản hồi động: Thay vì ngắt kết nối khi nhận thấy dấu hiệu rà quét, honeypot tiếp tục duy trì phiên làm việc. Nó đưa ra các banner dịch vụ giả mạo hoặc cung cấp một giao diện dòng lệnh ảo (fake shell) có cấu trúc giống hệt hệ thống thật. Điều này khiến tin tặc lầm tưởng rằng chúng đã xâm nhập thành công.
- Kích thích khai thác sâu: Hệ thống chủ động trả về các thông báo lỗi có chủ đích. Ví dụ, khi nhận được một ký tự nháy đơn ('), giao diện web sẽ trả về một thông báo lỗi cú pháp cơ sở dữ liệu. Cơ chế này dụ dỗ hacker tin rằng

ứng dụng thực sự có lỗ hổng, kích thích chúng tốn thêm thời gian để thử nghiệm các payload SQL Injection tinh vi hơn.

- **Kìm hãm và giữ chân:** Cơ chế này cố tình làm chậm tốc độ phản hồi của các kết nối mạng ở mức độ giao thức (TCP/IP). Việc phản hồi "nhỏ giọt" sẽ khiến các công cụ dò quét tự động hoặc bản thân kẻ tấn công bị treo trong trạng thái chờ đợi kéo dài. Cơ chế này vừa giúp hệ thống có thêm thời gian thu thập dữ liệu hành vi, vừa làm cạn kiệt tài nguyên thời gian của botnet mà không gây ra bất kỳ rủi ro nào cho mạng nội bộ.

1.3.3. Cơ chế thu thập và trích xuất dữ liệu

Trong các hệ thống Honeypot hiện đại, dữ liệu thu thập được đóng vai trò quan trọng trong việc phân tích hành vi tấn công và xây dựng các mô hình phát hiện xâm nhập. Khác với phương pháp ghi log truyền thống chỉ lưu trữ thông tin sau khi sự kiện xảy ra, các hệ thống Honeypot thường áp dụng cơ chế thu thập dữ liệu chủ động nhằm ghi nhận đầy đủ quá trình tương tác giữa đối tượng truy cập và môi trường môi nhử.

Dữ liệu được thu thập có thể bao gồm địa chỉ IP nguồn, thời gian truy cập, giao thức sử dụng, phương thức yêu cầu, thông tin định danh trình duyệt, nội dung request và các payload được gửi tới hệ thống. Các thông tin này phản ánh tương đối đầy đủ hành vi của đối tượng truy cập, từ đó hỗ trợ quá trình nhận diện các hoạt động quét dò, khai thác lỗ hổng hoặc thực hiện tấn công tự động.

Sau khi được ghi nhận, dữ liệu thường trải qua quá trình tiền xử lý nhằm loại bỏ các thông tin không cần thiết, chuẩn hóa định dạng và chuyển đổi về dạng phù hợp cho việc lưu trữ hoặc phân tích. Tùy theo mục đích sử dụng, dữ liệu có thể được biểu diễn dưới dạng tệp nhật ký, cơ sở dữ liệu hoặc các định dạng có cấu trúc như JSON.

Một yêu cầu quan trọng đối với cơ chế thu thập dữ liệu là đảm bảo tính toàn vẹn và độ tin cậy của thông tin ghi nhận được. Do đó, dữ liệu thường được lưu trữ tách biệt khỏi môi trường Honeypot nhằm hạn chế nguy cơ bị sửa đổi hoặc xóa bỏ khi hệ thống môi nhử bị tấn công. Việc duy trì tính toàn vẹn của dữ liệu giúp nâng cao độ chính xác trong quá trình phân tích, đồng thời tạo ra nguồn dữ liệu có giá trị cho các nghiên cứu và ứng dụng Machine Learning trong lĩnh vực an toàn thông tin.

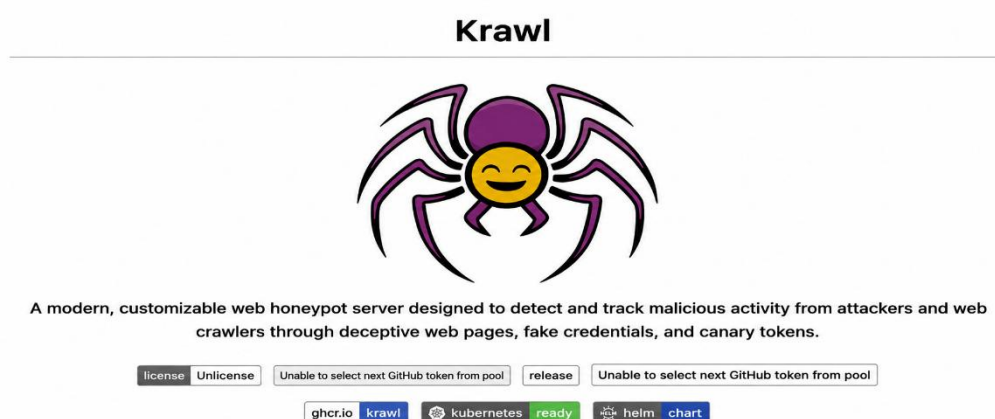
1.4. Tìm hiểu về Krawl

1.4.1. Giới thiệu tổng thể

Krawl là một nền tảng Web Honeypot và công nghệ đánh lừa mã nguồn mở, được thiết kế theo kiến trúc cloud-native. Chức năng chính của Krawl là tạo ra các ứng dụng web giả mạo một cách chân thực, cố tình chứa những điểm yếu bảo mật

để bị khai thác cùng các dữ liệu môi nhử. Mục tiêu của nó là phát hiện, theo dõi và ghi lại các hoạt động từ kẻ tấn công, bot và quét tự động (crawlers/scrapers) hay các phần mềm độc hại.

Được đóng gói gọn nhẹ dưới dạng container Docker, Krawl cung cấp khả năng triển khai nhanh chóng trên các môi trường Windows, Linux (như máy ảo Ubuntu) đi kèm với bảng điều khiển giám sát thời gian thực. Trong phạm vi của đề tài, Krawl được lựa chọn làm nền tảng cốt lõi vì khả năng chuyên biệt trong việc mô phỏng ứng dụng web và tự động hóa việc ghi nhận payload độc hại, từ đó tạo ra nguồn dữ liệu đầu vào cực kỳ giá trị và sạch nhiễu để phục vụ cho các mô hình Machine Learning sau này.



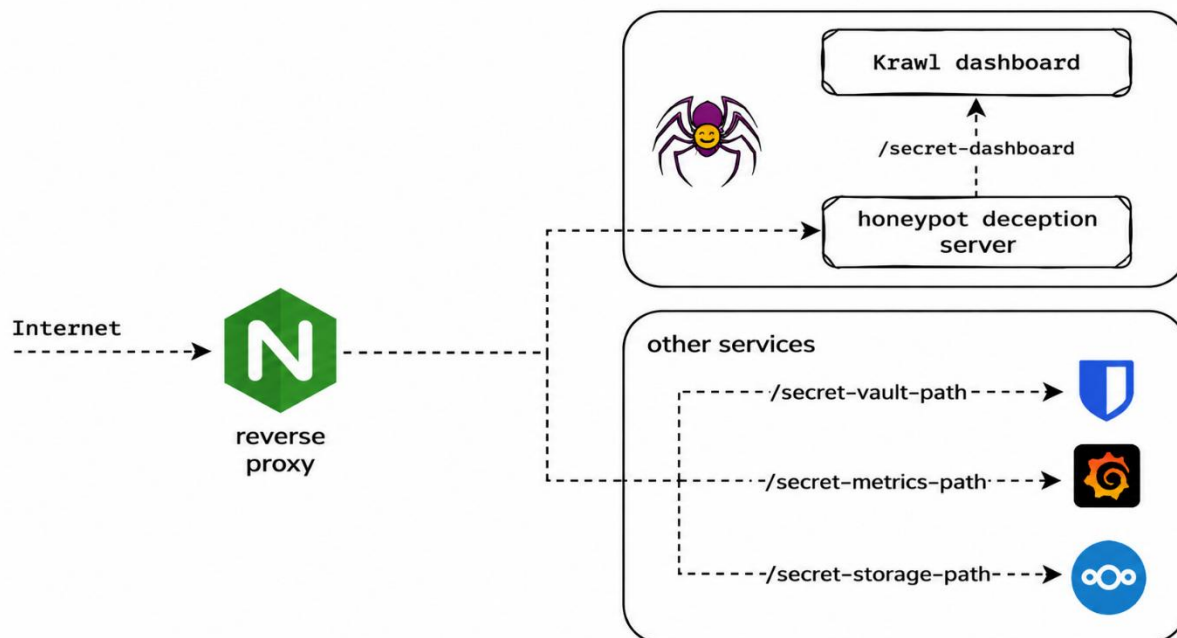
Hình 1.3 Krawl honeypot framework

1.4.2. Nguyên lý hoạt động

Cơ chế hoạt động của Krawl không thiên về việc ngăn chặn, mà xoay quanh nguyên lý "đánh lừa và ghi nhận hành vi". Quy trình này diễn ra qua các bước cốt lõi sau:

- Xây dựng mạng lưới môi nhử: Thay vì ẩn giấu, Krawl tự động sinh ra các "điểm mù" hấp dẫn tin tặc. Hệ thống tạo ra các đường dẫn ảo (ví dụ: /admin, /api/v1/secrets, /.env, /credentials.txt) và nhúng các dữ liệu cấu trúc giả vào mã nguồn HTML. Kẻ tấn công hoặc bot khi phân tích file robots.txt thường sẽ bị dụ dỗ truy cập ngay vào các thư mục được đánh dấu cấm này.
- Thu thập và phân tích tương tác: Khi kẻ tấn công bắt đầu tương tác (ví dụ: rà quét thư mục, thử đăng nhập brute-force, hoặc chèn các đoạn mã SQL Injection, Command Injection vào các form giả mạo), Krawl sẽ âm thầm tiếp nhận kết nối. Mọi chi tiết bao gồm HTTP Headers, User-Agent, địa chỉ IP, đường dẫn URI và toàn bộ chuỗi payload thực thi đều được ghi lại chi tiết vào web server log.

- Cách ly an toàn: Toàn bộ quá trình tương tác trên được diễn ra trong một môi trường ảo hóa cô lập hoàn toàn. Không có bất kỳ cơ sở dữ liệu thật hay dịch vụ nghiệp vụ nào được kết nối ở phía sau hệ thống. Dữ liệu log thu được sẽ được định dạng chuẩn hóa, sẵn sàng để các đoạn mã Python và Bash script trích xuất, làm sạch trước khi đẩy vào mô hình Machine Learning.



Hình 1.4 Một số chức năng của Krawl

1.4.3. Ưu điểm và hạn chế

Ưu điểm:

- Triển khai linh hoạt, tiêu tốn ít tài nguyên: Với kiến trúc chạy trên Docker, Krawl rất nhẹ, phù hợp để sinh viên triển khai thực nghiệm trên các máy ảo cá nhân mà không đòi hỏi cấu hình phần cứng phức tạp.
- Dữ liệu thu thập có độ trung thực cao: Vì bản chất là hệ thống môi nhử không phục vụ người dùng hợp lệ, mọi kết nối đến Krawl mặc định được xem là bất thường. Điều này giúp loại bỏ gần như hoàn toàn tỷ lệ dương tính giả. Log thu được rất "sạch", là nguồn dữ liệu lý tưởng để huấn luyện các thuật toán học máy.
- Giả lập cấu trúc chân thực: Khả năng tự sinh ra các API giả mạo và cấu trúc HTML tự nhiên giúp Krawl dễ dàng đánh lừa được nhiều loại công cụ rà quét lỗ hổng phổ biến, giữ chân kẻ tấn công đủ lâu để thu thập trọn vẹn payload.

Hạn chế:

- Giới hạn ở mức độ tương tác: Krawl tập trung giả lập ở tầng ứng dụng (giao thức HTTP/HTTPS), không cung cấp một hệ điều hành thực sự với quyền

shell đầy đủ (như các High-interaction Honeypot). Do đó, hệ thống khó bắt được các hành vi leo thang đặc quyền sâu ở tầng hệ thống.

- Nguy cơ bị nhận diện: Dù đã giả lập khá tốt, nhưng nếu các template web hoặc file mẫu nào không được người quản trị tùy biến lại mà dùng cấu hình mặc định, các bot AI tiên tiến của hacker có thể ghi nhớ khuôn mẫu và né tránh Krawl trong các đợt quét tiếp theo.

1.5. Tổng quan học máy

1.5.1. Học máy là gì?

Machine Learning (ML), hay còn gọi là học máy, là một nhánh của trí tuệ nhân tạo (AI) tập trung vào việc cho phép máy tính tự học từ dữ liệu mà không cần được lập trình rõ ràng cho từng tác vụ cụ thể. Thay vì phải viết từng dòng lệnh để máy tính thực hiện một công việc, Machine Learning sử dụng thuật toán để phân tích dữ liệu, nhận diện mẫu, và đưa ra dự đoán hoặc quyết định dựa trên những gì đã học được.

Hiểu một cách đơn giản, Machine Learning là quá trình máy tính học từ dữ liệu, giống như cách con người học hỏi từ kinh nghiệm. Ví dụ, một hệ thống Machine Learning có thể được huấn luyện trên một tập dữ liệu hình ảnh để nhận diện chó, mèo, hoặc các vật thể khác. Sau khi được huấn luyện, hệ thống có thể tự động phân loại các hình ảnh mới mà không cần sự can thiệp trực tiếp của con người.

Học máy thường bị nhầm lẫn với trí tuệ nhân tạo và học sâu. Trí tuệ nhân tạo là khái niệm bao quát nhất, không chỉ dừng lại ở các hệ thống có thể thực hiện những tác vụ đơn giản mà còn hướng đến việc giải quyết các vấn đề phức tạp, ra quyết định và học hỏi từ môi trường xung quanh. Trong khi đó, tổng quan về Machine Learning, đây là một nhánh nhỏ hơn của AI, tập trung vào việc đào tạo máy tính thông qua dữ liệu. Thay vì lập trình để thực hiện từng nhiệm vụ cụ thể, học máy cho phép hệ thống tự cải thiện và đưa ra quyết định dựa trên kinh nghiệm tích lũy từ dữ liệu đầu vào.

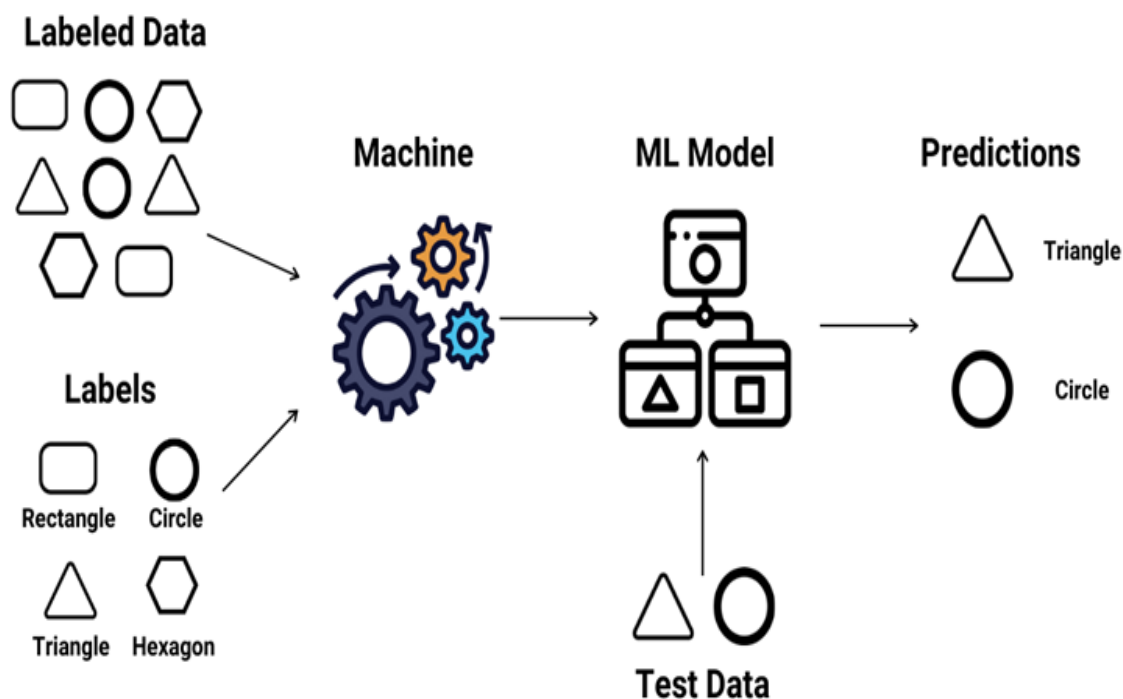
Mặc khác, học sâu, một phân nhánh của học máy, là cấp độ phức tạp và tiên tiến hơn, sử dụng mạng nơ-ron nhân tạo nhiều lớp để xử lý dữ liệu lớn và giải quyết các bài toán khó. Học sâu đặc biệt hiệu quả trong các lĩnh vực nhận diện hình ảnh, giọng nói và xử lý ngôn ngữ tự nhiên (NLP). Các công nghệ như xe tự lái, nhận diện khuôn mặt và dịch ngôn ngữ tự động đều dựa trên học sâu để hoạt động.

1.5.2. Các phương pháp học máy phổ biến

a) Học máy có giám sát

Học máy có giám sát là một trong những phương pháp học máy phổ biến và hiệu quả nhất hiện nay. Phương pháp này dựa trên việc sử dụng tập dữ liệu đã được

gắn nhãn để huấn luyện thuật toán, cho phép mô hình phân loại hoặc dự đoán kết quả với độ chính xác cao. (Hình 1.5)



Hình 1.5 Học máy có giám sát

Cách thức hoạt động của học có giám sát khá đơn giản nhưng mang lại hiệu quả mạnh mẽ. Thuật toán được cung cấp một tập dữ liệu bao gồm các đặc trưng và nhãn. Dựa trên mối quan hệ giữa dữ liệu đầu vào và đầu ra, mô hình dần học cách đưa ra dự đoán chính xác khi tiếp xúc với dữ liệu mới.

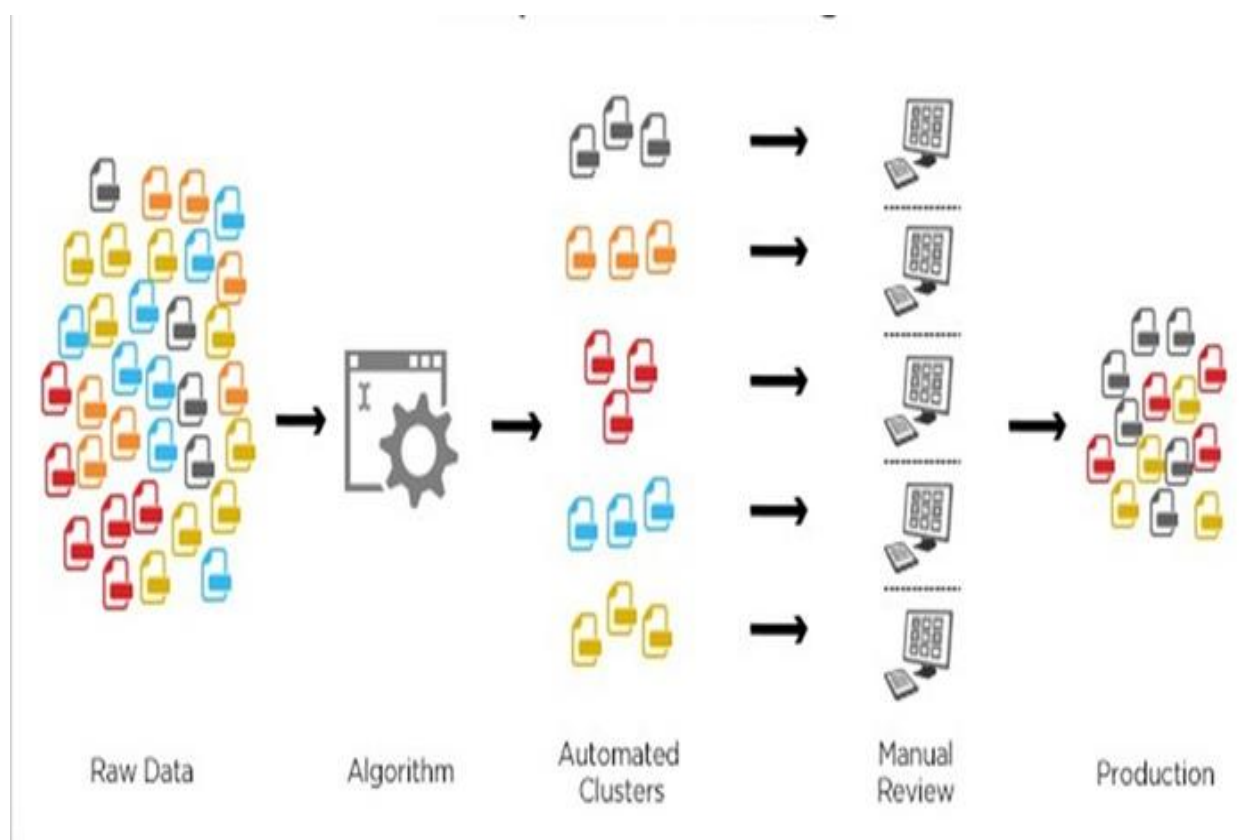
Dưới đây là một số thuật toán được sử dụng rộng rãi trong mô hình học máy có giám sát:

- Neural Networks: Mô phỏng hoạt động của não bộ, đặc biệt hữu ích trong xử lý hình ảnh và ngôn ngữ.
- Mô hình phân lớp Naive Bayes: Dự đoán xác suất dựa trên lý thuyết Bayes, hiệu quả với bài toán phân loại văn bản.
- Hồi quy tuyến tính: Dự đoán giá trị liên tục dựa trên mối quan hệ tuyến tính giữa các biến đầu vào và đầu ra.
- Hồi quy logistic: Thường được sử dụng cho các bài toán phân loại nhị phân (ví dụ: spam hay không spam).
- Random Forest: Tập hợp nhiều cây quyết định, tăng cường độ chính xác và giảm thiểu hiện tượng overfitting.
- Máy hỗ trợ vectơ (SVM): Tìm ra đường biên tối ưu để phân loại dữ liệu với khoảng cách lớn nhất giữa các nhóm dữ liệu.

b) Học máy không giám sát

Học máy không giám sát là một phương pháp dùng các thuật toán để phân tích và nhóm dữ liệu không có nhãn. Các thuật toán này tự động nhận biết và tìm ra những mẫu hoặc cụm ẩn trong dữ liệu mà không cần con người can thiệp. (Hình 1.6)

Chính nhờ khả năng tìm ra những đặc điểm chung và sự khác biệt trong dữ liệu, phương pháp này rất hiệu quả trong việc phân tích dữ liệu thăm dò, phát triển chiến lược bán chéo, phân khúc khách hàng, và nhận diện hình ảnh hay mẫu. Bên cạnh đó, học máy không giám sát còn giúp giảm số lượng tính năng trong mô hình thông qua các kỹ thuật giảm kích thước.



Hình 1.6 Học máy không giám sát

Ngoài ra, trong học không giám sát, còn có các thuật toán khác như mạng nơ-ron nhân tạo, thuật toán phân cụm K-means, và phương pháp phân cụm xác suất. Những thuật toán này giúp phân tích và nhóm dữ liệu lại với nhau dựa trên các đặc điểm chung, tìm ra những mô hình hoặc cấu trúc tiềm ẩn mà không cần nhãn đầu ra.

c) Học máy bán giám sát

Học bán giám sát là phương pháp học máy kết hợp học có giám sát và học không giám sát, tận dụng điểm mạnh của cả hai phương pháp để giải quyết các bài toán phức tạp. Quá trình đào tạo của phương pháp này sử dụng một tập dữ liệu có nhãn nhỏ hơn so với học có giám sát, kết hợp với một tập dữ liệu lớn không nhãn để

giúp mô hình học cách phân loại hoặc trích xuất tính năng từ dữ liệu không được gán nhãn.

Trong phương pháp học bán giám sát, mô hình bắt đầu với một tập dữ liệu có nhãn nhỏ, giúp hướng dẫn phân loại và tạo ra các quyết định chính xác cho một số ít trường hợp. Sau đó, thuật toán tiếp tục sử dụng tập dữ liệu không nhãn lớn hơn để mở rộng và cải thiện khả năng tổng quát của mô hình.

d) Học máy tăng cường

Học máy tăng cường là một phương pháp học máy, trong đó một hệ thống học cách hành động trong một môi trường thông qua việc thử nghiệm và rút kinh nghiệm. Mỗi lần hệ thống thực hiện một hành động, nó sẽ nhận được phản hồi từ môi trường dưới dạng phần thưởng hoặc hình phạt. Mục tiêu của phương pháp này là học cách hành động sao cho nhận được phần thưởng nhiều nhất có thể trong suốt quá trình học.

1.5.3. Ưu và nhược điểm của Machine Learning

a) Ưu điểm

Lợi ích của Machine Learning rất đa dạng nhờ vào khả năng tự động hóa và cải tiến không ngừng. Cụ thể, một số ưu điểm nổi bật của công nghệ này bao gồm:

- Nhận dạng mẫu: Máy học có khả năng phân tích và nhận diện các xu hướng, mẫu hình trong dữ liệu qua thời gian. Khi các thuật toán Machine learning được huấn luyện với nhiều dữ liệu hơn thì khả năng phát hiện các mối quan hệ trong dữ liệu trở nên chính xác hơn.
- Tự động hóa: Máy học có thể loại bỏ những công việc lặp đi lặp lại và tẻ nhạt, giúp tiết kiệm nguồn nhân lực và thời gian. Các công nghệ như tự động hóa quy trình bằng robot có thể thực hiện các công việc đơn giản như chọn và đóng gói sản phẩm trên dây chuyền lắp ráp. Bên cạnh đó, học máy còn giúp phát hiện gian lận và đánh giá mức độ rủi ro bảo mật liên tục, giúp bảo vệ dữ liệu và ngăn ngừa các rủi ro trước khi chúng trở thành vấn đề nghiêm trọng.
- Cải tiến liên tục: Một trong những ưu điểm nổi bật của học máy là khả năng cải tiến liên tục. Khi có thêm dữ liệu và thuật toán được tinh chỉnh, mô hình học máy sẽ ngày càng chính xác hơn và nhanh chóng hơn.

b) Nhược điểm

Bên cạnh những ưu điểm thì mô hình Machine learning cũng gặp phải một số hạn chế đáng chú ý:

- Thu thập lượng dữ liệu lớn: Học máy cần một lượng lớn dữ liệu để hoạt động tốt. Tuy nhiên, thu thập và chuẩn bị dữ liệu có thể rất khó khăn, đặc

biệt khi dữ liệu nằm rải rác ở nhiều nơi. Quá trình làm sạch dữ liệu, loại bỏ thông tin không cần thiết và chuẩn hóa dữ liệu cũng mất rất nhiều thời gian và công sức. Nếu dữ liệu không đủ chất lượng thì mô hình học máy sẽ không hoạt động chính xác.

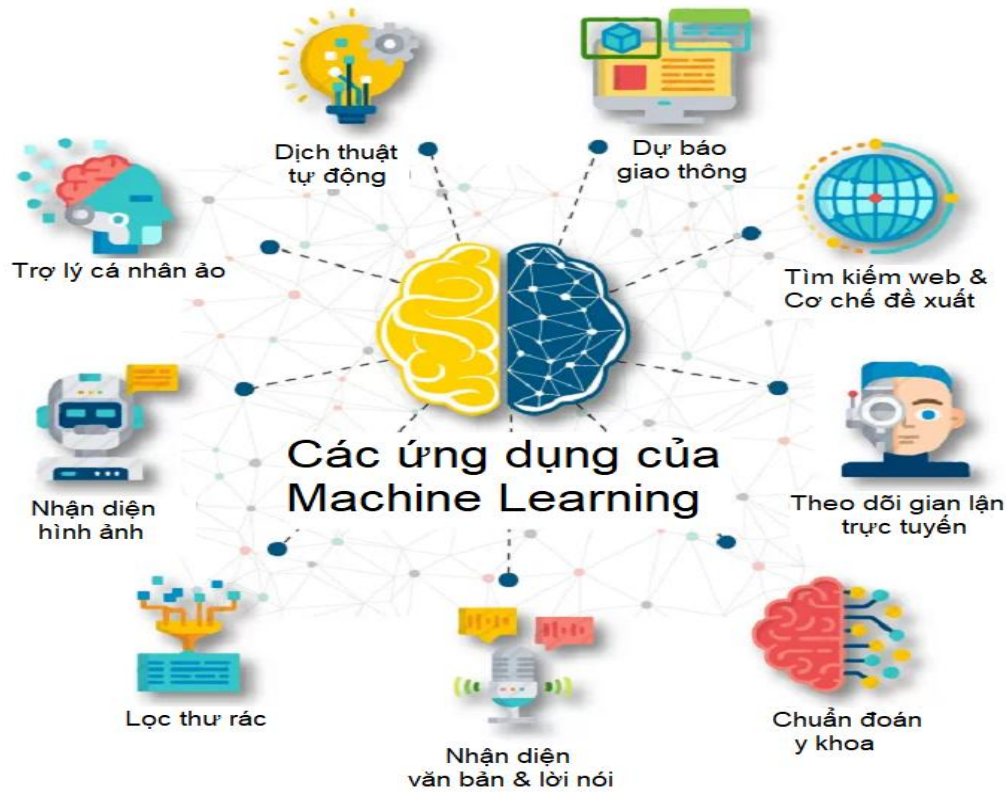
- Yêu cầu chuyên môn kỹ thuật: Mặc dù công nghệ học máy đang dần trở nên dễ tiếp cận hơn, nhưng các tổ chức vẫn cần đến các chuyên gia như lập trình viên hoặc nhà khoa học dữ liệu để triển khai và sử dụng các thuật toán. Người dùng cần phải hiểu cách các mô hình hoạt động và làm thế nào để tối ưu hóa chúng. Do vậy, Machine Learning đòi hỏi yêu cầu chuyên môn kỹ thuật khá cao.
- Tài nguyên chuyên sâu: Việc huấn luyện mô hình có thể rất tốn kém, đặc biệt khi làm việc với dữ liệu lớn. Các mô hình phức tạp cần phần cứng mạnh mẽ, như máy chủ có nhiều bộ vi xử lý hoặc bộ xử lý đồ họa, và đội ngũ nhân lực để xử lý dữ liệu và tối ưu hóa mô hình. Điều này có thể khiến học máy trở nên đắt đỏ và khó áp dụng với những doanh nghiệp không có đủ tài nguyên.

1.5.4. Ứng dụng thực tế

Học máy đang ngày càng trở nên phổ biến và được ứng dụng phổ biến ở nhiều lĩnh vực khác nhau. Dưới đây là các ứng dụng của Machine Learning trong thực tế:

- Y tế và chăm sóc sức khỏe: Học máy giúp chẩn đoán bệnh tật bằng cách phân tích hình ảnh y tế như X-quang, MRI, hay CT scan. Các mô hình học máy có thể nhận diện các dấu hiệu của bệnh ung thư, tim mạch hoặc các vấn đề sức khỏe khác một cách chính xác và nhanh chóng. Ngoài ra, học máy còn được sử dụng trong việc dự đoán bệnh nhân có nguy cơ cao mắc bệnh hoặc phát triển phác đồ điều trị cá nhân hóa.
- Dịch vụ khách hàng và chatbot: Các ứng dụng học máy như chatbot và trợ lý ảo đang được sử dụng rộng rãi để cung cấp dịch vụ khách hàng tự động. Học máy giúp các chatbot hiểu và phản hồi các câu hỏi của khách hàng một cách chính xác và tự nhiên hơn, giảm thiểu sự phụ thuộc vào nhân viên hỗ trợ.
- An ninh mạng: Máy học được sử dụng để phát hiện các mối đe dọa và hành vi gian lận trong hệ thống an ninh mạng. Các thuật toán học máy có thể phân tích lưu lượng mạng và hành vi của người dùng để nhận diện các dấu hiệu tấn công, giúp bảo vệ dữ liệu và ngăn ngừa các cuộc tấn công mạng.
- Tiếp thị và quảng cáo: Học máy được áp dụng để phân tích hành vi khách hàng và tối ưu hóa chiến lược tiếp thị. Các công ty có thể sử dụng học máy để cá nhân hóa quảng cáo, từ đó tăng hiệu quả chiến dịch quảng bá sản phẩm.

- phẩm. Bằng cách phân tích dữ liệu khách hàng, học máy giúp các doanh nghiệp hiểu rõ hơn về sở thích và thói quen mua sắm của người tiêu dùng.
- Xử lý ngôn ngữ tự nhiên (NLP): Đây là một trong các ứng dụng của Machine Learning. Học máy hỗ trợ các ứng dụng NLP như dịch máy hay nhận diện giọng nói. Các dịch vụ như Google Translate và Siri sử dụng học máy để hiểu và trả lời các câu hỏi từ người dùng bằng ngôn ngữ tự nhiên.



Hình 1.7 Ứng dụng của ML

1.6. Tổng kết chương 1

Chương 1 đã phác họa bức tranh tổng quan về tính cấp thiết của việc bảo vệ ứng dụng web trong thời đại bùng nổ chuyển đổi số. Từ việc phân tích những mặt hạn chế của các hệ thống phòng thủ thụ động truyền thống, chương đã làm nổi bật vai trò không thể thiếu của phương pháp phòng thủ chủ động bằng công nghệ Honeypot.

Bằng cách đi sâu tìm hiểu khái niệm, cách thức phân loại, kỹ thuật giảng bày thu thập log, và đặc biệt là phân tích chi tiết nền tảng giả lập ứng dụng web Krawl, nhóm thực hiện đã xây dựng được nền tảng kiến thức công nghệ vững chắc. Đây là tiền đề quan trọng để chuyển sang các nội dung tiếp theo: Nghiên cứu bản chất các lỗ hổng web, thực hành thu thập dữ liệu thô và tiến hành tích hợp các thuật toán Machine Learning nhằm tự động hóa quy trình phân tích hành vi tấn công sẽ được trình bày chi tiết ở các chương sau.

CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG HONEYPOT TÍCH HỢP MACHINE LEARNING

2.1. Phân tích yêu cầu và kiến trúc của hệ thống.

Xây dựng hệ thống phát hiện và phân tích hành vi tấn công mạng thông qua việc triển khai Krawl Honeypot nhằm thu hút, ghi nhận và bẫy các hoạt động quét mạng, botnet hoặc hành vi xâm nhập trái phép. Dữ liệu thu thập được sẽ được kết hợp với các thuật toán Machine Learning để tự động phân loại, nhận diện và cảnh báo các mối đe dọa trực tuyến theo thời gian thực, góp phần nâng cao hiệu quả giám sát và năng lực ứng phó với các cuộc tấn công mạng.

2.1.1. Mục tiêu cụ thể

Đề tài tập trung triển khai Krawl Honeypot nhằm xây dựng một môi trường cô lập có khả năng thu hút, ghi nhận và lưu trữ các hành vi tương tác của đối tượng tấn công. Thay vì chỉ phát hiện sự xuất hiện của các truy cập bất thường, hệ thống hướng tới việc thu thập dữ liệu có ngữ cảnh từ nhiều giai đoạn trong chuỗi tấn công mạng, đặc biệt là giai đoạn trinh sát và khai thác. Các hành vi được ghi nhận có thể bao gồm quét cổng, dò tìm dịch vụ, thu thập thông tin hệ thống, thử đăng nhập, gửi payload độc hại, khai thác lỗ hổng Web hoặc thực hiện các yêu cầu bất thường tới hệ thống.

Bên cạnh việc thu thập dữ liệu, đề tài ứng dụng các thuật toán Machine Learning nhằm hỗ trợ xử lý, phân tích và phân loại dữ liệu log thu được từ Krawl Honeypot. Dữ liệu sau khi được làm sạch, chuẩn hóa và trích xuất đặc trưng sẽ được sử dụng để huấn luyện các mô hình học máy. Các đặc trưng như request, payload, phương thức truy cập, user-agent, địa chỉ IP, tần suất tương tác và một số thuộc tính mạng khác có thể được khai thác để nhận diện các nhóm hành vi như truy cập bình thường, quét dò, injection, brute force, khai thác lỗ hổng hoặc các dạng tấn công khác.

Thông qua việc kết hợp Krawl Honeypot với Machine Learning, hệ thống hướng tới giảm sự phụ thuộc vào phương pháp phát hiện dựa trên chữ ký cố định, vốn thường gặp hạn chế trước các biến thể tấn công mới hoặc payload đã bị biến đổi. Mô hình học máy có thể học các đặc trưng tổng quát của hành vi tấn công, từ đó hỗ trợ phát hiện những request có dấu hiệu bất thường hoặc có khả năng gây nguy hiểm. Kết quả của quá trình phân loại sẽ góp phần hỗ trợ quản trị viên trong việc giám sát, phân tích log, đánh giá mức độ nguy cơ và đưa ra cảnh báo phù hợp đối với các mối đe dọa mạng.

2.1.2. Các chức năng chính

Chức năng bẫy và giả lập ứng dụng Web:

- Krawl Honeypot có nhiệm vụ giả lập một ứng dụng Web hoặc một số endpoint mới nhằm thu hút các tiến trình quét tự động, botnet hoặc đối tượng tấn công. Hệ thống có thể được cấu hình để lắng nghe trên các cổng phổ biến như HTTP 80/8080 hoặc HTTPS 443, đồng thời mô phỏng các đường dẫn thường bị dò quét như /wp-admin, /phpmyadmin, trang đăng nhập quản trị nội bộ hoặc giao diện quản trị của Router/CCTV.
- Khi nhận được request từ bên ngoài, Krawl Core phản hồi bằng các mã trạng thái HTTP phù hợp như 200 OK, 403 Forbidden hoặc 500 Internal Server Error, kết hợp với một số HTTP Header giả lập như thông tin máy chủ, phiên bản dịch vụ hoặc nền tảng vận hành. Mục đích của cơ chế này là tạo cảm giác rằng hệ thống đang tồn tại thật và có khả năng chứa các thành phần lỗi thời, từ đó kéo dài quá trình tương tác của đối tượng tấn công để phục vụ việc thu thập dữ liệu.

Chức năng ghi vết chuyên sâu

- Một chức năng quan trọng của Krawl Honeypot là ghi nhận chi tiết các thông tin liên quan đến mỗi request gửi đến hệ thống. Các dữ liệu cơ bản bao gồm địa chỉ IP nguồn, thời gian truy cập, phương thức HTTP như GET, POST, PUT, DELETE, đường dẫn request, phiên bản HTTP và mã phản hồi của hệ thống.
- Bên cạnh đó, Krawl còn thu thập các thành phần quan trọng trong HTTP Header như User-Agent, Cookie, Referer, Host và một số trường liên quan khác. Đặc biệt, hệ thống ghi lại phần dữ liệu được gửi lên thông qua Query String hoặc POST Body, nơi thường xuất hiện các payload độc hại như chuỗi kiểm thử SQL Injection, mã XSS, lệnh khai thác lỗ hổng hoặc đoạn mã dùng để tải Web Shell. Các thông tin này được lưu trữ có cấu trúc, ví dụ dưới định dạng JSON, nhằm tạo thuận lợi cho quá trình xử lý và phân tích ở các bước tiếp theo.

Chức năng trích xuất đặc trưng và học máy

- Sau khi dữ liệu log được thu thập, hệ thống tiến hành tiền xử lý và trích xuất các đặc trưng phục vụ cho mô hình Machine Learning. Các đặc trưng có thể bao gồm độ dài request, số lượng ký tự đặc biệt, sự xuất hiện của các từ khóa đáng ngờ, loại phương thức HTTP, thông tin User-Agent, tần suất truy cập, số lần request lỗi và các mẫu hành vi lặp lại từ cùng một nguồn.
- Dữ liệu sau khi được chuẩn hóa sẽ được đưa vào các thuật toán học máy như Decision Tree, Random Forest hoặc XGBoost để phân loại hành vi truy cập. Mô hình có thể hỗ trợ nhận diện các nhóm hành vi như truy cập bình thường, quét dò, injection, brute force, khai thác lỗ hổng Web hoặc các request bất thường khác. Kết quả phân loại giúp hệ thống tự động đánh giá mức độ nguy hiểm của từng request, giảm khối lượng phân tích thủ công và hỗ trợ quản trị viên trong quá trình giám sát an ninh mạng.

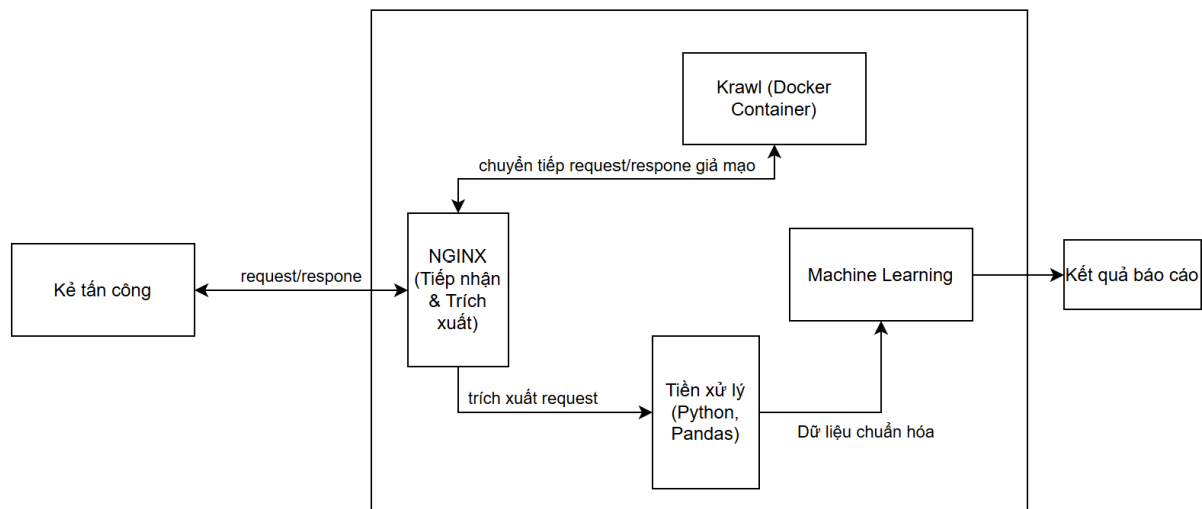
Chức năng cảnh báo và hỗ trợ phân tích:

- Dựa trên kết quả phân loại từ mô hình học máy, hệ thống có thể đưa ra cảnh báo đối với các request có dấu hiệu nguy hiểm hoặc bất thường. Nội dung cảnh báo có thể bao gồm thời gian phát hiện, địa chỉ IP nguồn, loại hành vi nghi ngờ, payload liên quan và mức độ tin cậy của mô hình dự đoán.
- Ngoài ra, dữ liệu thu thập từ Krawl Honeypot còn có thể được sử dụng để thống kê xu hướng tấn công, xác định các IP có tần suất truy cập bất thường, phân tích loại payload phổ biến và hỗ trợ xây dựng bộ dữ liệu phục vụ cho việc huấn luyện, đánh giá hoặc cải thiện mô hình phát hiện tấn công trong tương lai.

2.1.3. Mô hình kiến trúc

Nhằm tối ưu hóa khả năng phòng thủ chủ động và tự động nhận diện tấn công, hệ thống được thiết kế theo mô hình Đường ống dữ liệu kết hợp Trạm trung chuyển.

Thay vì ghi log một cách bị động, hệ thống chủ động chặn bắt toàn bộ lưu lượng mạng ngay từ tuyến đầu. Các HTTP Request thô sẽ di chuyển theo một chiều qua các bộ lọc xử lý và biến đổi thành các dữ liệu tình báo an toàn thông tin. Về mặt logic, kiến trúc hệ thống được chia thành 4 phân lớp chính.



Hình 2.1 Mô hình kiến trúc hệ thống

a) Lớp giao tiếp mạng và chặn bắt dữ liệu

Đây là cửa ngõ duy nhất của hệ thống giao tiếp với Internet hoặc mạng cục bộ, đóng vai trò như một màng lọc dữ liệu chủ động.

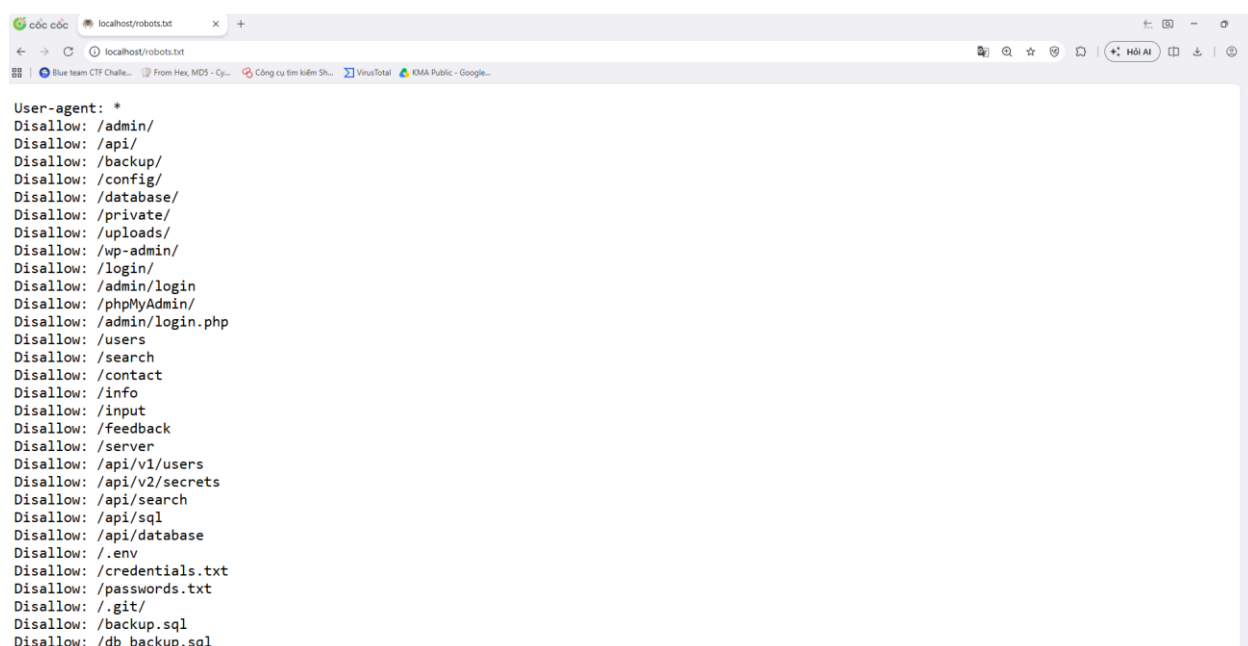
Lớp này hoạt động theo cơ chế toàn bộ lưu lượng mạng từ kẻ tấn công hoặc botnet nhắm vào hệ thống đều phải đi qua lớp này. Thay vì lưu trữ log thành các tệp tin tĩnh, lớp này thực hiện cơ chế bóc tách lưu lượng theo thời gian thực. Nó trích xuất toàn bộ cấu trúc của một HTTP Request bao gồm: Địa chỉ IP, HTTP Method,

URI, Headers và đặc biệt là phần thân – nơi chứa mã khai thác cốt lõi. Dữ liệu sau khi trích xuất sẽ được phân nhánh: một bản sao đẩy sang đường ống phân tích Học máy, bản gốc được chuyển tiếp vào lớp môi nhử.

b) Lớp môi trường môi nhử giả lập

Đóng vai trò là sân khấu ảo, cung cấp các mục tiêu giả mạo để thu hút và giữ chân kẻ tấn công.

Lớp này tiếp nhận các Request gốc được chuyển tiếp từ lớp Giao tiếp mạng. Tại đây, hệ thống sinh ra các ứng dụng web ảo, các endpoint API giả và các lỗ hổng cố ý. Khi kẻ tấn công tương tác, hệ thống sẽ trả về các phản hồi đánh lừa như giao diện đăng nhập thất bại, thông báo lỗi cơ sở dữ liệu giả.



Hình 2.2 Endpoint giả mạo

Lớp này được thiết kế theo nguyên tắc cô lập hoàn toàn. Mọi tương tác của hacker dù tinh vi đến đâu (như Remote Code Execution) cũng chỉ xảy ra trong môi trường ảo, không có đường dẫn ngược để tác động đến hệ điều hành lõi của máy chủ.

c) Lớp tiền xử lý dữ liệu

Lớp tiền xử lý dữ liệu đóng vai trò trung gian giữa hệ thống thu thập dữ liệu và mô hình Machine Learning. Nhiệm vụ chính của lớp này là chuẩn hóa dữ liệu thu thập được từ môi trường Honeypot, loại bỏ các thông tin không cần thiết và chuyển đổi dữ liệu về dạng phù hợp cho quá trình phân tích.

Thông qua lớp tiền xử lý, dữ liệu thô từ các HTTP Request được biến đổi thành tập đặc trưng có cấu trúc, giúp nâng cao chất lượng dữ liệu đầu vào và cải thiện hiệu quả hoạt động của các mô hình học máy.

d) Lớp phân tích học máy và cảnh báo

Đóng vai trò là bộ não phân tích của toàn bộ hệ thống, thay thế cho con người trong việc rà soát thủ công.

Dữ liệu sau khi số hóa được nạp vào các mô hình Học máy đã qua giai đoạn huấn luyện. Dựa trên các đặc trưng đã học, hệ thống tự động:

- Phân loại: Nhận diện và gán nhãn cho request (ví dụ: xếp loại tấn công SQL Injection, XSS hay Command Injection).
- Gom cụm: Nhóm các dấu hiệu bất thường có hành vi tương đồng để phát hiện các mẫu tấn công lạ (Zero-day) hoặc các chiến dịch rà quét diện rộng.

Kết quả phân tích được chuyển thành các thông tin phục vụ giám sát an ninh mạng như cảnh báo, thống kê hoặc báo cáo tổng hợp. Thông qua đó, người quản trị có thể nhanh chóng nhận biết các hành vi nghi ngờ và đưa ra biện pháp xử lý phù hợp.

2.2. Triển khai hệ thống

2.2.1. Kiến trúc tổng thể của hệ thống

Dựa trên mô hình kiến trúc đã được đề xuất ở mục trước, nhóm tiến hành triển khai hệ thống thực nghiệm trên môi trường Windows với mục tiêu xây dựng một nền tảng Honeypot có khả năng thu thập dữ liệu tấn công Web và tự động phân tích bằng Machine Learning. Toàn bộ hệ thống được thiết kế theo hướng module hóa, trong đó mỗi thành phần đảm nhận một chức năng riêng biệt nhưng vẫn liên kết chặt chẽ với nhau thông qua luồng dữ liệu thống nhất.

Kiến trúc triển khai bao gồm các thành phần chính: máy chủ Windows, máy chủ web Nginx, môi trường Honeypot Krawl, module tiền xử lý dữ liệu, module Machine Learning và hệ thống lưu trữ kết quả. Việc phân tách thành các module độc lập giúp hệ thống dễ dàng mở rộng, bảo trì và nâng cấp trong tương lai.

a) Máy chủ Windows và môi trường vận hành

Hệ điều hành Windows được lựa chọn làm nền tảng triển khai hệ thống nhờ giao diện quản trị trực quan, khả năng tương thích cao với nhiều phần mềm phát triển và thuận tiện trong quá trình cài đặt, cấu hình. Toàn bộ các thành phần của hệ thống như Docker Desktop, Nginx, Krawl Framework và các thư viện Machine Learning được triển khai và quản lý trên môi trường Windows.

Ngoài vai trò là nền tảng vận hành, Windows còn cung cấp các công cụ quản lý hệ thống, giám sát tài nguyên và cấu hình mạng giúp quá trình triển khai và kiểm thử hệ thống diễn ra thuận lợi. Các cơ chế quản lý tiến trình, quản lý bộ nhớ và dịch vụ của Windows góp phần đảm bảo các thành phần trong hệ thống hoạt động ổn định trong suốt quá trình thực nghiệm.

Để hỗ trợ việc triển khai các thành phần dựa trên container, hệ thống sử dụng Docker Desktop trên Windows. Công cụ này cho phép tạo lập và quản lý các môi trường cô lập một cách hiệu quả, đồng thời giúp đơn giản hóa quá trình triển khai Krawl Framework và các dịch vụ liên quan. Việc sử dụng container cũng giúp đảm bảo tính nhất quán giữa các lần thử nghiệm, hạn chế các lỗi phát sinh do khác biệt về môi trường cấu hình.

Bên cạnh đó, Windows hỗ trợ tốt các công cụ phát triển và phân tích dữ liệu như Python, Visual Studio Code, Docker Desktop và các thư viện Machine Learning. Điều này giúp việc xây dựng, kiểm thử và đánh giá mô hình trở nên thuận tiện hơn. Khi cần mở rộng hoặc nâng cấp hệ thống, các thành phần có thể được cài đặt và cấu hình nhanh chóng mà không yêu cầu thay đổi lớn đối với kiến trúc tổng thể.

b) Máy chủ Nginx

Nginx được triển khai ở lớp ngoài cùng của hệ thống và đóng vai trò là điểm tiếp nhận lưu lượng truy cập từ bên ngoài. Đây là thành phần đầu tiên tương tác với người dùng hoặc các tác nhân tấn công khi gửi request đến hệ thống.

Một trong những lý do lựa chọn Nginx là khả năng xử lý đồng thời số lượng lớn kết nối với mức sử dụng tài nguyên thấp. Điều này đặc biệt quan trọng đối với các hệ thống Honeypot vì chúng có thể phải tiếp nhận lượng lớn request từ các công cụ quét lỗ hổng hoặc botnet tự động.

Ngoài chức năng tiếp nhận kết nối, Nginx còn được cấu hình như một Reverse Proxy nhằm chuyển tiếp lưu lượng đến môi trường Honeypot Krawl. Việc sử dụng Reverse Proxy giúp che giấu cấu trúc nội bộ của hệ thống, đồng thời tạo thêm một lớp trung gian hỗ trợ giám sát và quản lý lưu lượng mạng.

Thông qua cơ chế ghi log và các định dạng dữ liệu tùy chỉnh, Nginx có khả năng lưu lại các thông tin quan trọng của request như địa chỉ IP nguồn, phương thức HTTP, URI truy cập, User-Agent và các trường Header. Các thông tin này là nguồn dữ liệu quan trọng phục vụ cho quá trình phân tích hành vi tấn công ở các bước tiếp theo. (Hình 2.3)

```
log_format strict_ml_format escape=none '--- RAW REQUEST START ---\n'
    'method:$clean_method\n'
    'path:$clean_path\n'
    'query:$clean_query\n'
    'host:$clean_host\n'
    'user_agent:$clean_user_agent\n'
    'accept:$clean_accept\n'
    'accept_en:$clean_accept_en\n'
    'accept_lar:$clean_accept_lar\n'
    'cookie:$clean_cookie\n'
    'content_ty:$clean_content_ty\n'
    'content_le:$clean_content_le\n'
    'body:$clean_body\n'
    '--- RAW REQUEST END ---\n';
```

Hình 2.3 Cấu hình Nginx đọc request

c) Môi trường Honeypot Krawl

Krawl Framework là thành phần trung tâm của hệ thống Honeypot. Nền tảng này được triển khai bên trong các Docker Container độc lập nhằm đảm bảo tính cô lập và an toàn trong quá trình vận hành.

Mục tiêu chính của Krawl là tạo ra các tài nguyên giả lập có khả năng thu hút và giữ chân kẻ tấn công. Các tài nguyên này có thể bao gồm giao diện đăng nhập, API giả, thư mục quản trị, tệp cấu hình hoặc các endpoint được thiết kế để mô phỏng các ứng dụng Web thực tế.

Khi nhận được request từ Nginx, Krawl sẽ phản hồi bằng các nội dung được xây dựng sẵn nhằm tạo cảm giác rằng hệ thống đang tồn tại các lỗ hổng hoặc thông tin giá trị. Điều này khiến đối tượng tấn công tiếp tục gửi thêm các request khai thác, qua đó giúp hệ thống thu thập được nhiều dữ liệu hơn về hành vi và kỹ thuật tấn công.

Việc triển khai Krawl bên trong Docker mang lại nhiều lợi ích. Nếu một payload độc hại được gửi đến hoặc xảy ra hành vi khai thác ngoài dự kiến, ảnh hưởng sẽ chỉ giới hạn trong container mà không tác động trực tiếp đến hệ điều hành máy chủ. Điều này góp phần đảm bảo tính an toàn cho toàn bộ môi trường thực nghiệm.

d) Module tiền xử lý dữ liệu

Dữ liệu thu thập từ Honeypot thường tồn tại dưới dạng bán cấu trúc và chứa nhiều thông tin không cần thiết cho quá trình phân tích. Vì vậy, hệ thống triển khai một module tiền xử lý dữ liệu nhằm chuẩn hóa dữ liệu trước khi đưa vào các mô hình Machine Learning.

Module này được xây dựng bằng Python với sự hỗ trợ của các thư viện như Pandas và NumPy. Quá trình xử lý bao gồm nhiều bước khác nhau như lọc dữ liệu

lỗi, loại bỏ dữ liệu trùng lặp, chuẩn hóa định dạng văn bản và trích xuất các đặc trưng có giá trị phân loại.

Ngoài ra, hệ thống còn thực hiện nhận diện các từ khóa và mẫu cú pháp thường xuất hiện trong các cuộc tấn công Web. Những đặc trưng này được chuyển đổi sang dạng số nhằm tạo ra tập dữ liệu phù hợp cho các thuật toán học máy.

Việc xây dựng một quy trình tiền xử lý hiệu quả không chỉ giúp giảm nhiễu dữ liệu mà còn góp phần nâng cao độ chính xác của mô hình phân loại trong giai đoạn huấn luyện và dự đoán.

e) Module Machine Learning và hệ thống báo cáo

Module Machine Learning được xây dựng bằng Python kết hợp với thư viện Scikit-learn. Thành phần này có nhiệm vụ tiếp nhận dữ liệu đã được chuẩn hóa và thực hiện quá trình huấn luyện, đánh giá cũng như dự đoán.

Trong phạm vi đề tài, các mô hình học máy được sử dụng để nhận diện và phân loại các dạng tấn công Web phổ biến như SQL Injection, Cross-Site Scripting (XSS), Path Traversal và Command Injection. Bên cạnh đó, hệ thống còn hỗ trợ phân tích các request bất thường nhằm phát hiện các dấu hiệu tấn công chưa từng xuất hiện trước đó.

Kết quả phân tích được tổng hợp thành các báo cáo và cảnh báo, giúp người quản trị có thể nhanh chóng theo dõi tình trạng an ninh của hệ thống. Các thông tin như loại tấn công, thời gian xảy ra, địa chỉ IP nguồn và mức độ nguy hiểm đều được ghi nhận nhằm phục vụ cho quá trình đánh giá và xử lý sự cố.

Thông qua việc kết hợp Honeypot với Machine Learning, hệ thống không chỉ có khả năng thu thập dữ liệu tấn công mà còn hỗ trợ tự động hóa quá trình phân tích, góp phần nâng cao hiệu quả giám sát và phát hiện các mối đe dọa trên môi trường Web.

2.2.2. Cấu hình môi trường internet

Để Krawl Honeypot có thể thu thập được các hành vi quét dò và tấn công từ bên ngoài, hệ thống cần được triển khai trong môi trường có kết nối Internet. Máy chủ honeypot được cấu hình để lắng nghe trên các cổng dịch vụ Web phổ biến nhằm mô phỏng một ứng dụng Web đang hoạt động và có khả năng thu hút các request từ bot, công cụ quét tự động hoặc đối tượng tấn công.

Các request đi vào sẽ được Krawl Honeypot tiếp nhận, phản hồi bằng các endpoint giả lập và ghi lại thông tin chi tiết phục vụ cho quá trình phân tích. Việc mở cổng cần được thực hiện có kiểm soát, chỉ cho phép các dịch vụ cần thiết hoạt động để hạn chế rủi ro bảo mật.

Bên cạnh đó, honeypot nên được đặt trong một môi trường tách biệt như máy ảo, container hoặc vùng mạng riêng nhằm tránh ảnh hưởng đến các hệ thống thật trong mạng nội bộ. Các kết nối đi ra từ honeypot cũng cần được giới hạn để ngăn chặn khả năng hệ thống bị lợi dụng làm điểm trung gian cho các hành vi tấn công khác.

Ngoài ra, hệ thống cần đồng bộ thời gian bằng NTP để đảm bảo các mốc thời gian trong log được ghi nhận chính xác. Dữ liệu log sau khi thu thập sẽ được lưu trữ cục bộ hoặc chuyển sang máy chủ phân tích để phục vụ các bước tiền xử lý, trích xuất đặc trưng và phân loại bằng mô hình Machine Learning.

2.2.3. Xây dựng và cấu hình các kịch bản môi nhử

Sau khi cấu hình môi trường Internet, hệ thống tiến hành xây dựng các kịch bản môi nhử nhằm thu hút các hành vi quét dò, truy cập bất thường và khai thác lỗ hổng từ bên ngoài. Các kịch bản này được thiết kế để mô phỏng những dịch vụ Web hoặc endpoint thường bị kẻ tấn công nhắm tới, từ đó tạo điều kiện cho Krawl Honeypot ghi nhận nhiều dạng tương tác khác nhau.

Trước hết, hệ thống cấu hình các đường dẫn giả lập phổ biến như /login, /admin, /search... Đây là những endpoint thường xuất hiện trong quá trình dò quét tự động của bot hoặc công cụ scanning. Khi nhận request đến các đường dẫn này, honeypot phản hồi bằng giao diện hoặc thông báo giả lập để tạo cảm giác rằng dịch vụ đang tồn tại thật.

Bên cạnh việc giả lập endpoint, Krawl Honeypot còn được cấu hình các phản hồi HTTP phù hợp như 200 OK, 401 Unauthorized, 403 Forbidden hoặc 500 Internal Server Error. Các phản hồi này có thể đi kèm với HTTP Header giả lập, biểu mẫu đăng nhập hoặc thông báo lỗi nhằm kéo dài quá trình tương tác của đối tượng tấn công. Việc phản hồi có chủ đích giúp hệ thống thu thập thêm các thông tin như phương thức truy cập, user-agent, payload, cookie, query string và tần suất request.

Ngoài ra, một số kịch bản có thể được xây dựng để ghi nhận hành vi nhập tài khoản, gửi payload độc hại, thử tải tệp lên hệ thống hoặc khai thác các tham số trong URL. Tuy nhiên, các chức năng này chỉ được mô phỏng trong môi trường cô lập, không cho phép thực thi mã độc hoặc ảnh hưởng đến hệ thống thật. Toàn bộ dữ liệu thu được từ các kịch bản môi nhử sẽ được lưu trữ dưới dạng log có cấu trúc để phục vụ quá trình phân tích và phân loại bằng Machine Learning.

Việc xây dựng các kịch bản môi nhử giúp honeypot tăng khả năng thu hút các tương tác có giá trị, đồng thời cung cấp nguồn dữ liệu thực tế hơn về hành vi của bot, công cụ quét tự động và các đối tượng tấn công trên Internet.

2.3. Thu thập và xử lý dữ liệu

2.3.1. Thu thập dữ liệu

Dữ liệu phục vụ cho hệ thống được thu thập trực tiếp từ quá trình hoạt động của Krawl Honeypot. Khi honeypot được triển khai trên môi trường Internet, các request từ bot quét tự động, công cụ scanning hoặc đối tượng tấn công sẽ được ghi nhận và lưu trữ lại dưới dạng log. Đây là nguồn dữ liệu chính dùng để phân tích hành vi và huấn luyện mô hình Machine Learning. Qua quá trình thu thập ban đầu ta thu thập được 70 nghìn file dữ liệu gồm 4 loại nhãn File Attack, Injection, XSS, Normal (Theo hình 2.4).

Mỗi bản ghi log bao gồm các thông tin cơ bản như địa chỉ IP nguồn, thời gian truy cập, phương thức HTTP, đường dẫn request, mã phản hồi HTTP, user-agent, header, query string và payload gửi kèm. Đối với các request có nội dung đáng ngờ, hệ thống đặc biệt ghi nhận các dấu hiệu như chuỗi truy vấn bất thường, ký tự đặc biệt, mã script, câu lệnh SQL, đường dẫn nhảy cảm hoặc hành vi thử truy cập vào các endpoint quản trị.

Row Labels	Count of label
File_Attack	17499
Injection	17507
Normal	17509
XSS	17506
Grand Total	70021

Hình 2.4 Số lượng của từng nhãn thu thập được

Quá trình thu thập dữ liệu được thực hiện liên tục trong thời gian hệ thống honeypot hoạt động. Các log sau khi sinh ra sẽ được lưu dưới dạng có cấu trúc nhằm thuận tiện cho quá trình tiền xử lý, chuẩn hóa và trích xuất đặc trưng ở các bước tiếp theo. Việc lưu trữ có cấu trúc cũng giúp dễ dàng thống kê số lượng request, phân tích tần suất truy cập, xác định các IP có hành vi bất thường và phân loại các dạng tấn công phổ biến.

Bên cạnh dữ liệu thu thập trực tiếp từ Krawl Honeypot, hệ thống có thể bổ sung hoặc đối chiếu với các mẫu request đã được gán nhãn nhằm hỗ trợ quá trình xây

dựng bộ dữ liệu huấn luyện. Sau khi được tổng hợp, dữ liệu sẽ được kiểm tra, loại bỏ các bản ghi trùng lặp hoặc thiếu thông tin quan trọng trước khi chuyển sang giai đoạn xử lý và phân tích bằng Machine Learning.

2.3.2. Tiền xử lý dữ liệu

Dữ liệu được sử dụng trong đề tài được thu thập từ các HTTP Request gửi đến môi trường Honeypot Krawl thông qua máy chủ Nginx. Các request này phản ánh trực tiếp hành vi tương tác của người dùng hoặc đối tượng tấn công với hệ thống, đồng thời chứa nhiều thông tin quan trọng phục vụ cho quá trình nhận diện và phân loại tấn công.

Tuy nhiên, dữ liệu thu thập được ban đầu tồn tại dưới dạng log hoặc chuỗi ký tự thô, trong đó nhiều trường dữ liệu chưa phù hợp để sử dụng trực tiếp cho các thuật toán Machine Learning. Do đó, cần thực hiện quá trình tiền xử lý nhằm chuẩn hóa dữ liệu, giảm nhiễu và xây dựng tập đặc trưng phục vụ cho việc huấn luyện mô hình.

a) Làm sạch và chuẩn hóa dữ liệu thô

Trong quá trình thu thập, hệ thống có thể ghi nhận nhiều loại request khác nhau với cấu trúc và độ dài không đồng nhất. Một số request có thể thiếu trường dữ liệu, chứa ký tự đặc biệt hoặc phát sinh lỗi trong quá trình truyền tải. Những bản ghi như vậy có thể ảnh hưởng đến chất lượng của tập dữ liệu nếu không được xử lý phù hợp.

Tiếp theo, các trường thông tin cốt lõi cấu thành một request như phương thức HTTP (GET, POST, PUT...), đường dẫn URI, nội dung request và payload được đưa về một định dạng chuẩn hóa duy nhất. Đối với dữ liệu văn bản, hệ thống thực hiện một chuỗi các kỹ thuật xử lý bao gồm:

- Chuyển đổi chữ thường: Đưa toàn bộ chuỗi ký tự về dạng chữ thường để triệt tiêu sự khác biệt ý nghĩa giữa các từ viết hoa/viết thường (ví dụ: SELECT, Select và select sẽ được hiểu là một).
- Giải mã hóa URL: Các cuộc tấn công như SQL Injection hay Cross-Site Scripting (XSS) thường mã hóa ký tự dạng Hex (như %27 cho dấu nháy đơn, %3C cho dấu <). Hệ thống sẽ tiến hành giải mã các chuỗi này về dạng ký tự gốc để mô hình nhận diện chính xác hành vi bất thường.
- Loại bỏ ký tự thừa: Loại bỏ các khoảng trắng dư thừa, các ký tự điều khiển ngắt dòng (\n, \r) không mang giá trị phân loại.
- Ghép chuỗi: Các thành phần riêng lẻ nhưng có mối quan hệ logic của request sẽ được ghép lại thành một chuỗi văn bản thống nhất, tạo tiền đề cho việc phân tích ngữ cảnh toàn diện.

Sau quá trình chuẩn hóa, tập dữ liệu thu được có tính nhất quán cao hơn và phản ánh chính xác các hoạt động diễn ra trên môi trường Honeypot.

b) Trích xuất đặc trưng số

Bởi vì các thuật toán Machine Learning bản chất là các mô hình toán học và chỉ có thể tính toán trên các ma trận số, bước tiếp theo hệ thống phải thực hiện chuyển đổi chuỗi văn bản đã chuẩn hóa thành tập hợp các đặc trưng số

Một số đặc trưng được sử dụng trong đề tài bao gồm: Độ dài URI, độ dài phần Body, số lượng tham số truy vấn, số lượng ký tự đặc biệt, tần suất xuất hiện của các ký tự mã hóa URL, sự xuất hiện của các từ khóa liên quan đến tấn công Web.

Ngoài các đặc trưng định lượng, hệ thống còn ghi nhận các mẫu cú pháp đặc trưng thường gặp trong các cuộc tấn công như SQL Injection, Cross-Site Scripting (XSS), Path Traversal và Command Injection.

Đối với các request chứa nội dung văn bản, hệ thống thực hiện quá trình nhận diện và chuẩn hóa các từ khóa quan trọng trước khi đưa vào mô hình học máy. Các từ khóa như SELECT, UNION, DROP, SCRIPT, ALERT hoặc các chuỗi liên quan đến truy cập tệp hệ thống được xem là những chỉ báo có giá trị trong quá trình phân tích.

Sau khi nhận diện, các từ khóa được chuyển đổi thành các thuộc tính số học thể hiện tần suất hoặc sự xuất hiện của chúng trong request. Cách biểu diễn này giúp mô hình học máy xử lý dữ liệu hiệu quả hơn và giảm ảnh hưởng của sự khác biệt về cách biểu diễn payload.

method	path_len	query_len	body_len	slash_count	dot_count	path_spec	query_spec	body_spec	digit_ratio	percent	ei	has_sql	has_sql_pattern	has_xss	has_xss_keywords	has_path	has_cmd	html_bracket_count	quote_count	js_bracket_count
0	11	3	0	2	0	2	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	11	20	0	2	0	2	1	0	0.09375	0	1	1	0	0	0	0	0	0	0	0
0	11	18	0	2	0	2	4	0	0	0	0	0	0	0	1	0	0	0	0	0
0	11	10	0	2	0	2	3	0	0	0	0	0	0	0	0	0	0	0	0	0
0	6	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	12	0	31	2	0	2	0	7	0.068182	0	1	1	0	0	0	0	0	0	3	0
0	7	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	11	18	0	2	0	2	4	0	0	0	0	0	0	0	1	0	0	0	0	0
0	11	26	0	2	0	2	7	0	0.026316	0	0	0	0	1	0	0	4	0	2	2
0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	12	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	13	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	16	0	0	1	0	1	0	0	0.125	0	0	0	0	0	0	0	0	0	0	0
0	6	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	12	0	31	2	0	2	0	7	0.068182	0	1	1	0	0	0	0	0	3	0	0
1	12	0	54	2	0	2	0	13	0.044776	0	1	1	1	0	0	4	3	2	2	2
1	12	0	48	2	0	2	0	9	0.016393	0	0	0	1	0	0	4	0	2	0	0
0	7	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	11	20	0	2	0	2	1	0	0.09375	0	1	1	0	0	0	0	0	0	0	0
0	11	21	0	2	0	2	1	0	0.121212	0	1	1	0	0	0	0	0	0	0	0
0	11	18	0	2	0	2	4	0	0	0	0	0	0	0	1	0	0	0	0	0
0	11	20	0	2	0	2	1	0	0.09375	0	1	1	0	0	0	0	0	0	0	0
0	11	20	0	2	0	2	1	0	0.09375	0	1	1	0	0	0	0	0	0	0	0
0	11	18	0	2	0	2	4	0	0	0	0	0	0	0	1	0	0	0	0	0
0	11	52	0	2	0	2	12	0	0.03125	2	0	0	0	0	0	0	3	0	0	0
0	11	49	0	2	0	2	13	0	0.04918	3	0	0	0	0	0	4	0	2	0	0
0	11	167	0	2	0	2	36	0	0	0	1	0	0	0	0	2	8	10	0	0
0	11	18	0	2	0	2	4	0	0	0	0	0	0	0	1	0	0	0	0	0
0	11	20	0	2	0	2	1	0	0.09375	0	1	1	0	0	0	0	0	0	0	0
0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Hình 2.5 Đặc trưng sau chuẩn hóa

Kết quả cuối cùng của quá trình tiền xử lý là một tập dữ liệu có cấu trúc, trong đó mỗi request được biểu diễn bằng một tập gồm 19 đặc trưng số. Tập dữ liệu này được sử dụng làm đầu vào cho các mô hình Machine Learning ở giai đoạn huấn luyện và đánh giá được trình bày trong các mục tiếp theo.

c) Sẵn sàng cho giai đoạn huấn luyện mô hình

Bộ dữ liệu sau khi trải qua các bước làm sạch, chuẩn hóa và biểu diễn dưới dạng số lúc này đã đạt trạng thái tối ưu để chuyển giao sang phân hệ huấn luyện. Dữ liệu này sẽ được phân tách theo tỷ lệ phù hợp (ví dụ 80% cho tập huấn luyện - Training Set và 20% cho tập kiểm thử - Testing Set).

Tập dữ liệu số hóa chất lượng cao này là đầu vào quyết định giúp các thuật toán học máy giám sát phát huy tối đa sức mạnh phân loại:

- + Decision Tree: Xây dựng các quy tắc rẽ nhánh tường minh, giúp phân loại nhanh các request dựa trên các ngưỡng ký tự đặc biệt.
- + Random Forest: Kết hợp nhiều cây quyết định độc lập để giảm thiểu hiện tượng quá khớp, tăng độ ổn định khi nhận diện các biến thể tấn công mới.
- + XGBoost: Thuật toán tối ưu hóa theo cơ chế Boosting, giúp tăng tối đa độ chính xác, tối ưu thời gian huấn luyện và có khả năng phân loại cực tốt trên các bộ dữ liệu log Honeypot có sự mất cân bằng lớn giữa request bình thường và request tấn công.

Mục tiêu cuối cùng của bộ dữ liệu tiền xử lý này là giúp các mô hình trên học được ranh giới phân tách hành vi, từ đó tự động phân loại chính xác các hành vi truy cập hợp lệ và phát hiện kịp thời các request bất thường, các cuộc dò quét độc hại hướng vào hệ thống Krawl Honeypot theo thời gian thực.

2.4. Xây dựng và huấn luyện mô hình machine learning

2.4.1. Bài toán đưa ra

Trong hệ thống phát hiện và phân tích hành vi tấn công mạng sử dụng Krawl Honeypot, dữ liệu thu thập được chủ yếu là các request HTTP và log tương tác giữa đối tượng truy cập với hệ thống môi nhử. Các request này rất đa dạng, bao gồm truy cập hợp lệ, hành vi quét dò, thử khai thác lỗ hổng, chèn mã độc hoặc các dạng truy cập bất thường khác. Do số lượng log sinh ra trong thực tế là rất lớn và khó có thể phân tích thủ công, đề tài đặt ra bài toán ứng dụng Machine Learning để tự động hóa quy trình phân loại hành vi nguy cơ.

Bài toán được xác định là bài toán phân loại đa lớp. Với đầu vào của mô hình là các đặc trưng được tạo ra từ nội dung request, payload, query string, user-agent hoặc các trường log liên quan. Đầu ra của mô hình là nhãn phân loại tương ứng, ví dụ như Normal, Injection, XSS, hoặc các nhóm tấn công khác tùy theo bộ dữ liệu được xây dựng.

Mục tiêu cốt lõi của bài toán là huấn luyện được một mô hình học máy có khả năng nhận diện chính xác các dấu hiệu độc hại hoặc bất thường từ các request. Kết quả dự đoán từ mô hình sẽ hỗ trợ đắc lực cho quản trị viên trong việc giám sát log tự động, phát hiện sớm nguy cơ và đưa ra các cảnh báo bảo mật kịp thời.

2.4.2. Lựa chọn thuật toán

Để giải quyết bài toán phân loại đa lớp đã định nghĩa ở trên, đề tài lựa chọn thực nghiệm và đánh giá ba thuật toán Machine Learning phổ biến dựa trên cấu trúc cây quyết định bao gồm: Decision Tree, Random Forest và XGBoost. Việc lựa chọn bộ ba thuật toán này nhằm mục đích so sánh hiệu quả giữa ba hướng tiếp cận kinh điển: mô hình đơn lẻ, mô hình học máy tổ hợp dạng Bagging và dạng Boosting.

Decision Tree được lựa chọn do có cấu trúc đơn giản, dễ triển khai và dễ diễn giải kết quả. Thuật toán này hoạt động bằng cách xây dựng các luật phân nhánh dựa trên đặc trưng của dữ liệu, từ đó đưa ra quyết định phân loại. Trong bài toán phát hiện tấn công mạng, Decision Tree giúp quan sát được cách mô hình phân biệt giữa các nhóm request khác nhau, tuy nhiên mô hình này có thể dễ bị quá khớp nếu dữ liệu phức tạp.

Random Forest được sử dụng nhằm khắc phục một phần hạn chế của Decision Tree. Thuật toán này kết hợp nhiều cây quyết định khác nhau để đưa ra kết quả dự đoán cuối cùng, nhờ đó giúp tăng độ ổn định và giảm nguy cơ quá khớp. Random Forest phù hợp với dữ liệu có nhiều đặc trưng, đồng thời cho kết quả khá tốt trong các bài toán phân loại hành vi bất thường.

XGBoost được lựa chọn vì đây là thuật toán boosting mạnh, có khả năng tối ưu hóa quá trình học và cải thiện hiệu quả phân loại. Khác với Random Forest, XGBoost xây dựng các cây quyết định theo từng bước, trong đó cây sau tập trung sửa lỗi cho cây trước. Nhờ đó, mô hình có thể học tốt hơn các mẫu dữ liệu phức tạp và cải thiện độ chính xác trong phân loại. Tuy nhiên, XGBoost thường yêu cầu tài nguyên tính toán cao hơn và cần điều chỉnh tham số phù hợp để đạt hiệu quả tốt.

Việc lựa chọn ba thuật toán trên nhằm mục đích so sánh hiệu quả giữa các hướng tiếp cận khác nhau: mô hình đơn giản, mô hình tổ hợp nhiều cây và mô hình boosting. Các mô hình sẽ được huấn luyện trên cùng một bộ dữ liệu, sau đó đánh giá bằng các chỉ số như Accuracy, Precision, Recall và F1-score để lựa chọn thuật toán phù hợp nhất cho hệ thống phát hiện và phân loại hành vi tấn công mạng.

2.4.3. Huấn luyện mô hình

Quy trình thực nghiệm được bắt đầu ngay sau khi hoàn tất giai đoạn tiền xử lý và trích xuất đặc trưng keyword-based. Lúc này, bộ dữ liệu tổng thể sẽ được phân chia thành hai phần độc lập: tập huấn luyện và tập kiểm thử. Tập huấn luyện đóng vai trò cung cấp dữ liệu mẫu để ba mô hình Decision Tree, Random Forest, XGBoost học cách phân biệt dấu hiệu của từng nhóm hành vi. Trong khi đó, tập kiểm thử được giữ lại hoàn toàn để đánh giá khả năng tổng quát hóa của mô hình trên những dữ liệu mới chưa từng xuất hiện trong quá trình học, đảm bảo tính khách quan cho kết quả thực nghiệm.

✓ 4. Train/Test split

```
[ ] X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

✓ 5. Train Random Forest Classifier

```
[ ] model = RandomForestClassifier(  
    n_estimators=200,      # số cây  
    max_depth=20,         # độ sâu  
    min_samples_split=5,  
    n_jobs=-1,            # dùng hết CPU  
    random_state=42  
)  
  
model.fit(X_train, y_train)
```

Hình 2.6 Quá trình huấn luyện mô hình ML

Để tạo cơ sở đối sánh hiệu năng một cách công bằng, quá trình huấn luyện được tiến hành tuần tự và độc lập cho từng thuật toán trên cùng một tập dữ liệu đầu vào. Các mô hình sẽ tập trung phân tích tần suất hoặc sự hiện diện của các từ khóa trọng yếu trong request HTTP để thiết lập mối quan hệ hàm số với nhãn phân loại.

Sau khi quá trình huấn luyện hoàn tất, các mô hình sẽ thực hiện dự đoán nhãn trên tập kiểm thử để trích xuất các chỉ số đánh giá. Mô hình tối ưu nhất sau đó sẽ được đóng gói và lưu trữ dưới dạng tệp định dạng .pkl (Pickle) để sẵn sàng đưa vào vận hành thực tế. (Hình 2.6)

Trong môi trường triển khai thực tế, quy trình xử lý dữ liệu được thiết lập theo cơ chế đồng bộ:

- Khi hệ thống nhận được một request HTTP mới từ Krawl Honeypot, request này ngay lập tức đi qua luồng tiền xử lý và trích xuất đặc trưng dựa trên từ khóa tương tự như giai đoạn huấn luyện.
- Các đặc trưng mới sinh ra được chuyển thẳng vào mô hình .pkl đã nạp sẵn trong bộ nhớ.
- Mô hình thực hiện tính toán tốc độ cao và trả về kết quả phân loại hành vi theo thời gian thực, giúp hệ thống đưa ra phản hồi hoặc cảnh báo bảo mật kịp thời.

2.4.4. Đánh giá kết quả

Sau khi hoàn thành quá trình huấn luyện, các mô hình Decision Tree, Random Forest và XGBoost được tiến hành đánh giá độc lập trên tập kiểm thử nhằm kiểm tra năng lực tổng quát hóa. Các chỉ số đo lường tiêu chuẩn được áp dụng bao gồm Accuracy, Precision, Recall và F1-score. Trong đó, Accuracy phản ánh độ chính xác tổng thể trên toàn bộ tập dữ liệu; Precision thể hiện tỷ lệ dự đoán chính xác trong số các mẫu được mô hình phân loại vào một nhãn hành vi; Recall đo lường khả năng phát hiện không bỏ sót các mẫu thực tế thuộc về nhãn đó; và F1-score đóng vai trò là chỉ số trung bình điều hòa giúp đánh giá sự cân bằng giữa Precision và Recall.

	precision	recall	f1-score	support
File_Attack	0.93	0.64	0.76	3509
Injection	0.63	0.93	0.75	3509
Normal	1.00	0.86	0.92	3502
XSS	0.97	0.82	0.89	3501
accuracy			0.80	14021
macro avg	0.88	0.81	0.83	14021
weighted avg	0.85	0.80	0.80	14021

Hình 2.7 Kết quả đánh giá mô hình Decision Tree

Kết quả đánh giá thực nghiệm (Theo hình 2.7) cho thấy mô hình cơ sở Decision Tree đạt độ chính xác tổng thể là 80% và chỉ số Macro F1-score đạt 83%. Mô hình này bộc lộ sự mất cân bằng khá lớn giữa chỉ số Precision và Recall ở các lớp tấn công. Điển hình là đối với hành vi Injection, mô hình đạt Recall lên tới 0.93 nhưng Precision chỉ đạt 0.63, cho thấy một lượng lớn các request thuộc lớp khác đang bị cây quyết định đơn lẻ nhận diện nhầm thành hành vi Injection.

	precision	recall	f1-score	support
File_Attack	0.94	0.98	0.96	3509
Injection	0.97	0.94	0.96	3509
Normal	0.99	0.98	0.99	3502
XSS	0.99	0.98	0.99	3501
accuracy			0.97	14021
macro avg	0.97	0.97	0.97	14021
weighted avg	0.97	0.97	0.97	14021

Hình 2.8 Kết quả đánh giá mô hình Random Forest

Tiếp theo, mô hình học máy tổ hợp Random Forest (Theo hình 2.8) được thử nghiệm trên cùng một tập dữ liệu trích xuất từ khóa. Nhờ cơ chế bỏ phiếu đồng thuận từ nhiều cây quyết định độc lập, mô hình đã ghi nhận sự tăng trưởng vượt trội về hiệu năng khi đạt độ chính xác tổng thể vọt lên 97% và Weighted F1-score đạt 97%. Đáng chú ý, Random Forest đã tối ưu hóa xuất sắc độ chính xác của các lớp Injection và XSS, đồng thời nhận diện phân loại chính xác gần như tuyệt đối đối với nhóm hành vi bình thường.

	precision	recall	f1-score	support
File_Attack	0.94	0.98	0.96	3509
Injection	0.97	0.93	0.95	3509
Normal	0.99	0.98	0.98	3502
XSS	0.98	0.99	0.99	3501
accuracy			0.96	14021
macro avg	0.97	0.97	0.97	14021
weighted avg	0.96	0.96	0.96	14021

Hình 2.9 Kết quả đánh giá mô hình XGBoost

Đối với mô hình XGBoost (Theo hình 2.9), kết quả thu được tương đương với Random Forest khi đạt Accuracy tổng thể 96% và Weighted F1-score 96%. Điều này cho thấy tập đặc trưng được xây dựng từ các thuộc tính cấu trúc của HTTP Request và các từ khóa tấn công đã cung cấp đủ thông tin để cả hai mô hình học và phân biệt các lớp dữ liệu với độ chính xác cao.

XGBoost duy trì khả năng phát hiện tấn công hiệu quả trên hầu hết các lớp, đặc biệt lớp File_Attack đạt Recall 0.98 và lớp XSS đạt Recall 0.99. Tuy nhiên, Precision của lớp File_Attack chỉ đạt 0.94, thấp hơn so với các lớp còn lại. Điều này cho thấy vẫn tồn tại một số mẫu thuộc các lớp khác bị dự đoán nhầm thành File_Attack, phản ánh sự chồng lấn nhất định giữa các đặc trưng của các loại tấn công trong bộ dữ liệu.

So sánh với Random Forest, hiệu năng của XGBoost không có sự cải thiện đáng kể. Nguyên nhân có thể xuất phát từ việc các đặc trưng đầu vào chủ yếu được xây dựng dựa trên tập từ khóa và các đặc trưng thống kê đơn giản của request HTTP. Trong không gian đặc trưng này, cả Random Forest và XGBoost đều đã khai thác được phần lớn thông tin phân biệt giữa các lớp, dẫn đến kết quả đạt mức tương đương nhau.

Bảng dưới đây tổng hợp kết quả đánh giá của ba mô hình:

Bảng 2: So sánh giữa 3 mô hình machine learning

Mô hình	Accuracy	F1-score	Weighted F1-score
Decision Tree	80%	83%	80%
Random Forest	97%	97%	97%
XGBoost	96%	97%	96%

2.5. Tích hợp mô hình vào hệ thống phân tích

Dựa trên kết quả đối sánh thực nghiệm, hai mô hình Random Forest và XGBoost đều thể hiện hiệu năng phân loại vượt trội với độ chính xác tổng thể đạt 96-97%, vượt xa mô hình cây quyết định đơn lẻ. Tuy nhiên, để tối ưu hóa khả năng tích hợp và vận hành theo thời gian thực trong hệ thống Krawl Honeypot, đề tài quyết định lựa chọn mô hình XGBoost làm bộ phân loại cốt lõi nhờ vào tính ổn định, tốc độ phản hồi nhanh đối với cấu trúc ma trận đặc trưng từ khóa và đặc biệt để giảm đi khả năng overfitting khi sử dụng.

Quy trình tích hợp và triển khai mô hình vào hệ thống phân tích luồng dữ liệu của Krawl Honeypot được thực hiện theo cấu trúc đồng bộ gồm các bước sau:

- Đóng gói mô hình: Mô hình tối ưu sau khi huấn luyện xong được chuyển đổi và lưu trữ thành tệp tin cấu hình định dạng định danh .pkl thông qua thư viện joblib. Việc này giúp đóng băng toàn bộ các trọng số, luật phân nhánh đã học được từ tập dữ liệu Train.

- Xây dựng module Tiền xử lý đồng bộ: Module này được nhúng trực tiếp vào luồng bắt log của Honeypot. Khi có một request HTTP mới tương tác với hệ thống mỗi nhữ, module sẽ thực hiện giải mã, chuẩn hóa khoảng trắng và trích xuất đúng 18 đặc trưng y hệt như quy trình xử lý dữ liệu huấn luyện.
- Nhận diện và Cảnh báo theo thời gian thực: Tập .pkl được nạp sẵn vào bộ nhớ hệ thống khi khởi động. Đặc trưng của request mới sau khi trích xuất sẽ được chuyển thẳng vào mô hình để thực thi lệnh dự đoán. Kết quả phân loại sẽ được trả về trong mili-giây, cho phép hệ thống Krawl Honeypot lập tức ghi log chi tiết hoặc đẩy cảnh báo nguy cơ trực tiếp lên giao diện quản trị viên.

2.6. Tổng kết chương 2

Tóm lại, Chương 2 đã trình bày một cách toàn diện và chi tiết quy trình xây dựng, thực nghiệm và đánh giá hệ thống phát hiện hành vi tấn công mạng dựa trên các thuật toán Học máy. Đề tài đã hoàn thành các mục tiêu nghiên cứu cốt lõi đặt ra trong chương bao gồm:

- Xác định bài toán: Định hình rõ ràng bài toán phân loại đa lớp với đầu vào là các trường dữ liệu thu thập từ log tương tác của Krawl Honeypot và đầu ra là các nhãn hành vi tấn công cụ thể.
- Cải tiến phương pháp trích xuất đặc trưng: Thay vì sử dụng các phương pháp tính toán trọng số phức tạp, đề tài đã tối ưu hóa bộ trích xuất đặc trưng dựa trên từ khóa kết hợp với việc bóc tách cấu trúc ký tự đặc biệt theo từng phân vùng. Sự điều chỉnh này giúp giảm tải tài nguyên tính toán nhưng vẫn giữ được các dấu hiệu nhận diện đặc trưng của các lớp tấn công.
- Thực nghiệm và Đối sánh khoa học: Quá trình huấn luyện tuần tự trên ba thuật toán cấu trúc cây đã tạo ra một bức tranh thực nghiệm trực quan. Kết quả cho thấy các mô hình tổ hợp đạt hiệu năng áp đảo với độ chính xác 83%, chứng minh tính đúng đắn của việc ứng dụng Ensemble Learning trong việc phân tích log an toàn thông tin.

CHƯƠNG 3 TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Thực nghiệm

3.1.1. Môi trường triển khai

Bảng 3 Thông số môi trường thực nghiệm

Thành phần	Thông số chi tiết
CPU	Intel Core i5-12500H
RAM	16 GB
Hệ điều hành	Windows 11 Home 64-bit
Môi trường ảo hóa	Docker Desktop
Môi trường phân tích	Python

Hệ thống phần cứng :

- Nền tảng môi nhử(Krawl Honeypot): Được triển khai linh hoạt dưới dạng container thông qua Docker Desktop trên máy vật lý nhằm đảm bảo môi trường bẫy được cô lập an toàn. Hệ thống được cấu hình ánh xạ trực tiếp ra các cổng dịch vụ mạng tiêu chuẩn (Port 80 và 443) để hứng chịu các luồng truy cập thật từ bên ngoài. Nhiệm vụ chính của nền tảng này là giả lập các đường dẫn/lỗ hổng web mục tiêu, giữ chân các công cụ rà quét tự động và ghi nhận toàn bộ thông tin (IP, HTTP Header, chuỗi Payload) vào file log hệ thống.
- Mô đun phân tích : Được xây dựng trên môi trường Python và chạy độc lập như một tiến trình nền trên hệ điều hành máy vật lý. Mô đun này được nạp sẵn tệp mô hình học máy XGBoost (.pkl) đã qua huấn luyện từ trước. Nó có nhiệm vụ liên tục tiếp nhận dữ liệu log thô do Honeypot sinh ra, tiến hành chuẩn hóa, trích xuất 18 đặc trưng từ khóa và tự động phân loại luồng dữ liệu theo thời gian thực để xác định xem đó là truy cập hợp lệ hay thuộc các nhóm tấn công (Injection, XSS, File Attack).

3.1.2. Các kịch bản thực nghiệm

Để đánh giá toàn diện khả năng thu thập của hệ thống môi nhử và độ chính xác của mô hình phân loại, quá trình thực nghiệm được tiến hành bằng cách giả lập các luồng dữ liệu mạng hỗn hợp. Hệ thống sẽ phải đối mặt với 4 kịch bản chính, bao gồm các truy cập hợp lệ và các hình thức tấn công web phổ biến nhất:

- Kịch bản 1 - Truy cập hợp lệ: Mô phỏng hành vi của người dùng thông thường. Kịch bản được thực hiện bằng cách sử dụng trình duyệt web để thao tác như một người dùng thật: truy cập trang chủ, chuyển hướng giữa các trang và tải các tài nguyên tĩnh mà không gửi kèm bất kỳ payload độc hại nào.
- Kịch bản 2 - Tấn công Path Traversal: Kịch bản này được thực hiện thủ công bằng cách lợi dụng các điểm yếu trên tham số URL để cố gắng truy cập trái phép vào các tệp tin hệ thống nằm ngoài thư mục gốc của máy chủ web. Cụ thể, quá trình này được tiến hành bằng cách gõ trực tiếp các chuỗi ký tự thao tác thư mục (ví dụ: ../, ..\) kết hợp với tên các tệp tin cấu hình nhạy cảm (như /etc/passwd đối với Linux hoặc boot.ini, win.ini đối với Windows) nhằm kiểm tra khả năng nhận diện từ khóa và dấu hiệu đặc trưng của lỗ hổng Path Traversal từ hệ thống học máy.
- Kịch bản 3 - Tấn công Injection: Tập trung vào việc khai thác các điểm yếu đầu vào thông qua SQL Injection (SQLi) và Command Injection (CMDi). Quá trình này được thực hiện bằng cách tự gõ các payload chứa ký tự đặc biệt và từ khóa thao tác cơ sở dữ liệu/hệ thống (ví dụ: ' OR 1=1 --, UNION SELECT user, password, ; whoami) trực tiếp vào các form đăng nhập, ô tìm kiếm hoặc truyền qua tham số trên URL.
- Kịch bản 4 - Tấn công XSS: Nhằm kiểm tra khả năng bắt lỗi các thẻ HTML và dấu ngoặc bất thường của mô hình. Kịch bản được tiến hành bằng cách cố tình chèn các đoạn mã JavaScript độc hại (ví dụ: `<script>alert(document.cookie)</script>`, `<img src=x onerror=prompt(1)>`) vào các trường nhập liệu có khả năng lưu trữ hoặc phản hồi lại dữ liệu trên giao diện.

3.2. Kết quả thu được

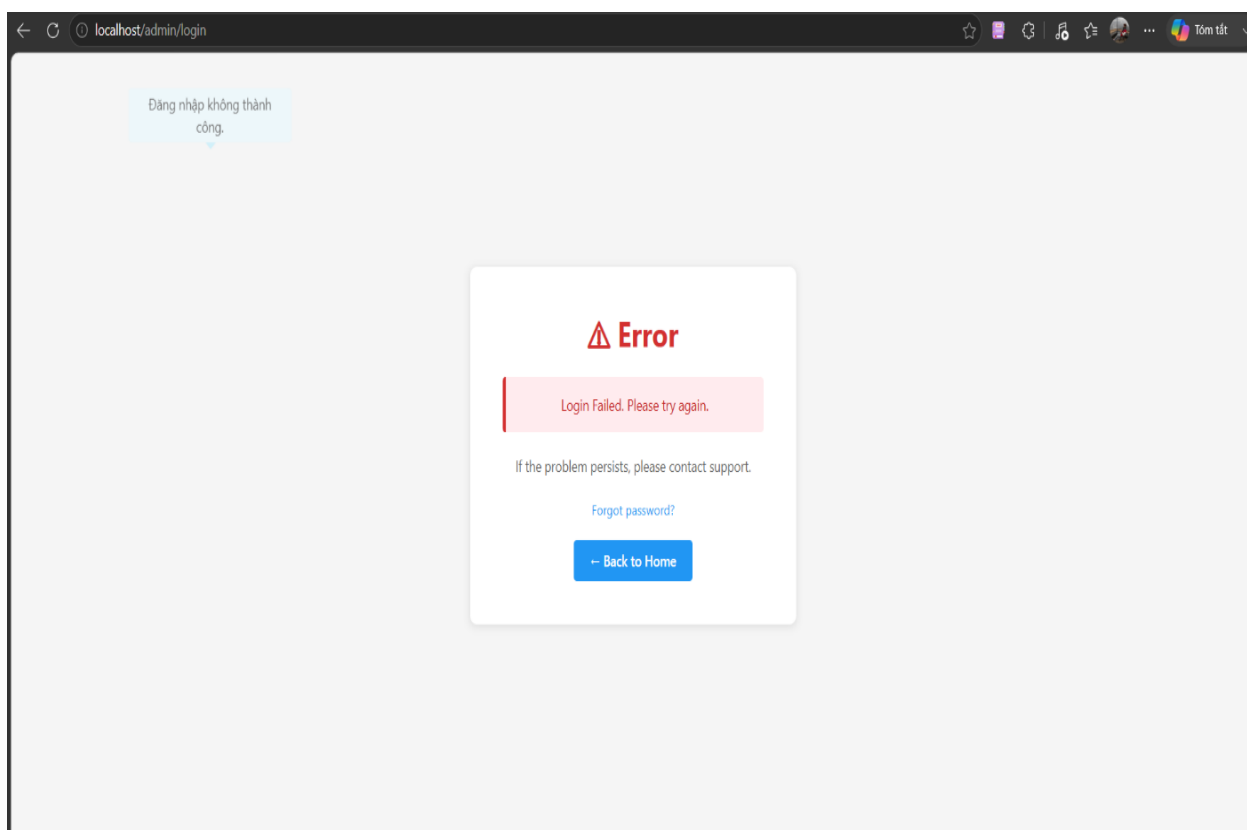
3.2.1. Đánh giá năng lực thu thập của Krawl Honeypot

Khả năng đánh lừa: Krawl Honeypot đã làm rất tốt nhiệm vụ "giữ chân" kẻ tấn công. Thông thường, đối với các hệ thống thực tế, khi xuất hiện các payload độc hại (như chèn mã HTML/JS hay truy cập thư mục ẩn), tường lửa hoặc máy chủ web sẽ lập tức từ chối bằng các mã lỗi như HTTP 403 Forbidden hoặc 404 Not Found. Tuy nhiên, hệ thống mô phỏng Krawl vẫn tiếp nhận và trả về mã trạng thái HTTP 200 OK kèm theo giao diện web ảo. Cơ chế này khiến kẻ tấn công hoặc các công cụ rà quét tự động lầm tưởng rằng lỗ hổng thực sự tồn tại, từ đó tiếp tục tương tác và gửi thêm các payload khai thác sâu hơn.

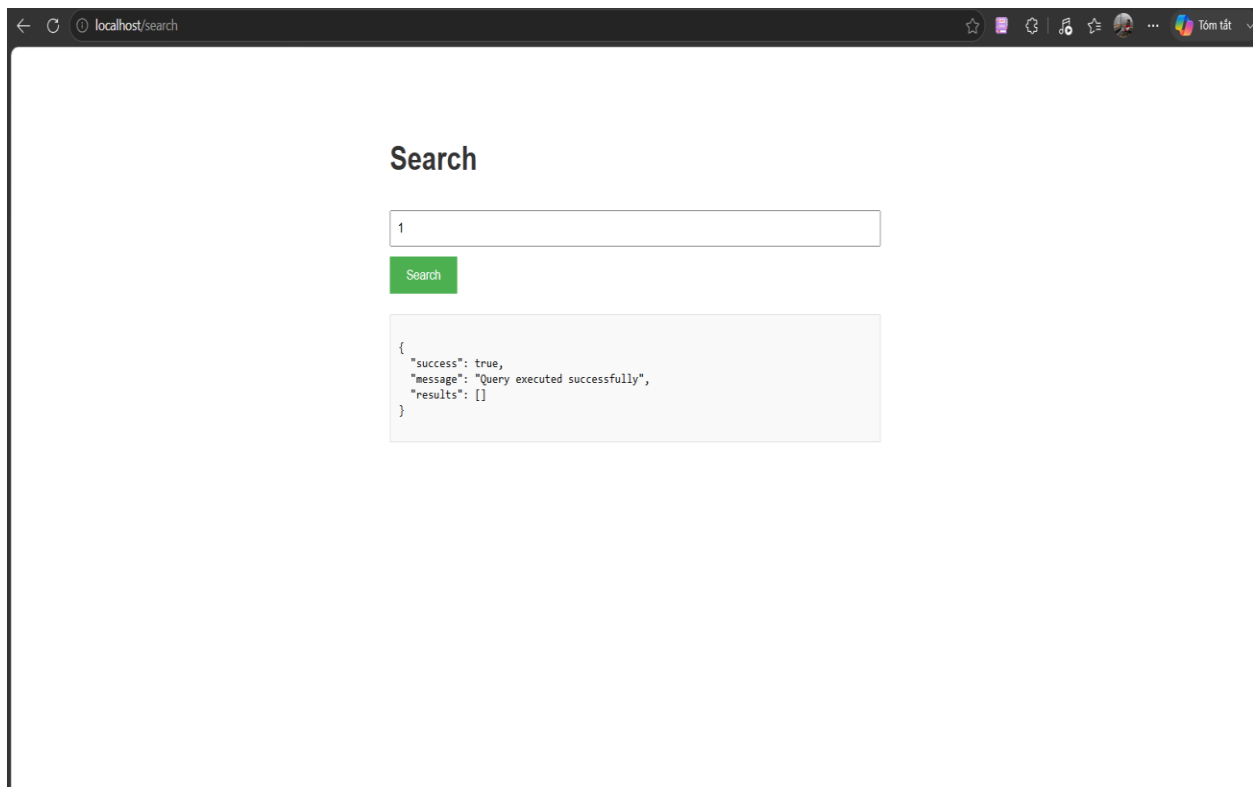
3.2.2. Hiệu năng phân loại của XGBoost

Sau khi Krawl Honeypot hoàn thành nhiệm vụ thu thập, mô đun phân tích dữ liệu (sử dụng ngôn ngữ Python) sẽ lập tức tiếp nhận tệp log để xử lý. Thực nghiệm cho thấy mô đun này hoạt động cực kỳ ổn định theo cơ chế thời gian thực. Quá trình đọc luồng log, giải mã (Hex, URL Decode), làm sạch văn bản và bóc tách 19 đặc trưng diễn ra với độ trễ rất thấp. Dưới đây là minh chứng chi tiết về khả năng trích xuất đặc trưng và gán nhãn của mô hình XGBoost đối với từng kịch bản cụ thể:

- Kịch bản 1: Truy cập hợp lệ: Hệ thống bóc tách thành công các request tải tài nguyên thông thường. Các chỉ số như tỷ lệ ký tự đặc biệt, số lượng dấu ngoặc đều nằm trong ngưỡng an toàn. Mô hình gán nhãn chính xác là *Normal*, đảm bảo không sinh ra các cảnh báo giả (False Positive) làm nhiễu quá trình giám sát của quản trị viên.
- + Hành vi truy cập: Người dùng thao tác lướt web thông thường như truy cập trang chủ, tải các tài nguyên tĩnh (HTML, CSS, hình ảnh) hoặc nhập các thông tin văn bản thuần túy không chứa mã độc.

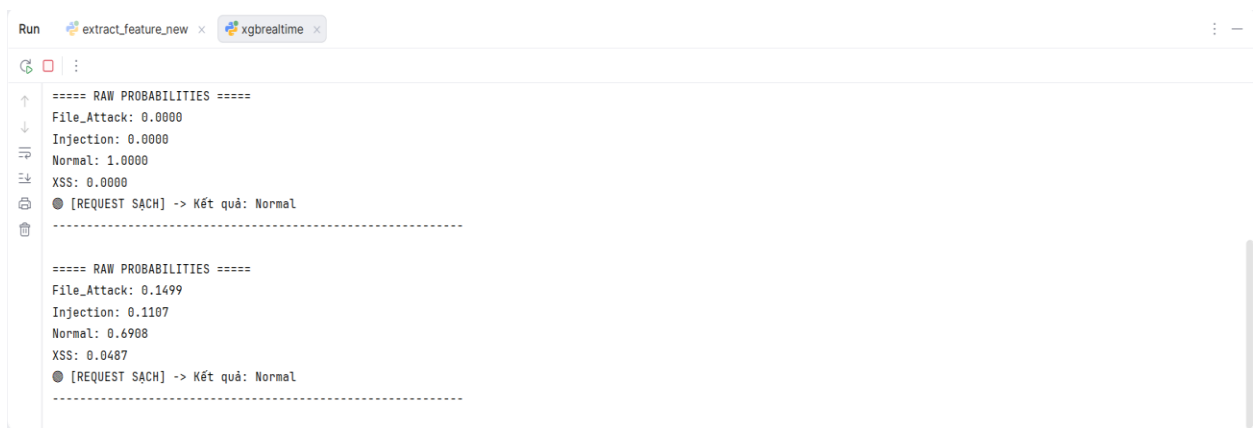


Hình 3.3 Kết quả hiển thị trên trình duyệt khi truy cập đường dẫn /login



Hình 3.4 Kết quả hiển thị trên trình duyệt khi truy cập đường dẫn /search

- + Hệ thống phát hiện: Tiến trình Python bóc tách thành công các request này. Nhờ các chỉ số đếm ký tự đặc biệt và cờ nhận diện từ khóa đều ở mức an toàn (giá trị 0), mô hình XGBoost phân loại và gán nhãn chính xác là Normal. Điều này chứng minh hệ thống không sinh ra các cảnh báo giả làm nhiễu quá trình giám sát



Hình 3.5 Hệ thống hiển thị phân loại luồng dữ liệu mang nhãn Normal

- Kịch bản 2: Tấn công duyệt thư mục (Path Traversal):
 - + Hành vi tấn công: Kẻ tấn công cố ý nhập các chuỗi ký tự thao tác thư mục kết hợp với tên tệp tin nhạy cảm của hệ thống (ví dụ: ../../../../etc/passwd hoặc boot.ini) trực tiếp vào ô tìm kiếm (Search box)

trên giao diện web hoặc truyền qua tham số URL nhằm thử nghiệm khả năng đọc tệp tin trái phép.

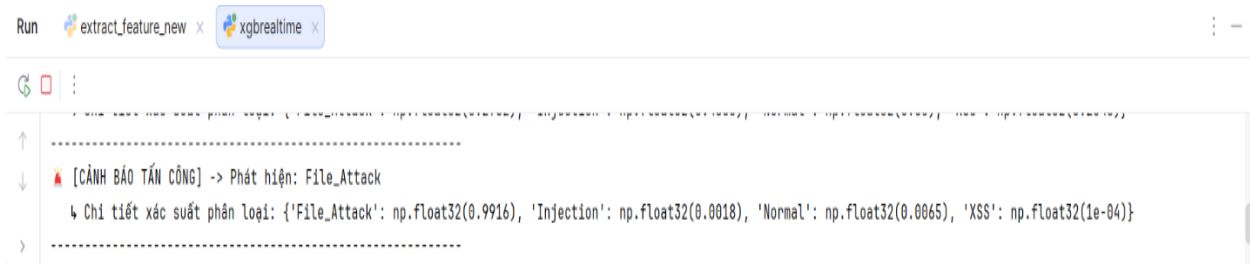
Search

Search

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
mysql:x:108:113:MySQL Server,,,:/nonexistent:/bin/false
sshd:x:109:65534:./run/sshd:/usr/sbin/nologin
api_backup:x:1899:1845:./home/admin_admin:/usr/bin/zsh
api_dev:x:1497:1920:./home/test:/bin/sh
monitor_service:x:1273:1241:./home/prod_dev:/bin/bash
```

Hình 3.6 Giao diện trình duyệt khi nhập payload Path Traversal

- + Hệ thống phát hiện: Mô đun xử lý bằng Python ngay lập tức bóc tách luồng log (phần tham số query/body). Thuật toán nhận diện thành công từ khóa nằm trong danh sách cấm, kích hoạt cờ `has_path_traversal` (đạt giá trị 1). Dựa trên trọng số đặc trưng này, mô hình XGBoost nhanh chóng phân loại và đưa ra cảnh báo luồng dữ liệu mang nhãn `File_Attack` trên màn hình giám sát.



Hình 3.7 Terminal hiển thị nhận diện và cảnh báo nhãn File_Attack

- Kịch bản 3 - Tấn công Injection:
 - + Hành vi tấn công: Tiến hành chen các chuỗi ký tự thao tác cơ sở dữ liệu phổ biến như ' OR 1=1 -- hoặc UNION SELECT vào các tham số truy vấn (query parameter), ô tìm kiếm hoặc gửi qua các form đăng nhập giả mạo của hệ thống Honeypot.

Search

```
SELECT * FROM administrators, users ORDER BY created_at DESC LIMIT 50;
```

Search

```
{
  "success": true,
  "message": "Query executed successfully",
  "results": []
}
```

Hình 3.8 Thao tác gửi payload SQL Injection vào ô tìm kiếm

Krawl Login
Please log in to continue

Username
a' or '1' = '1'

Password
.

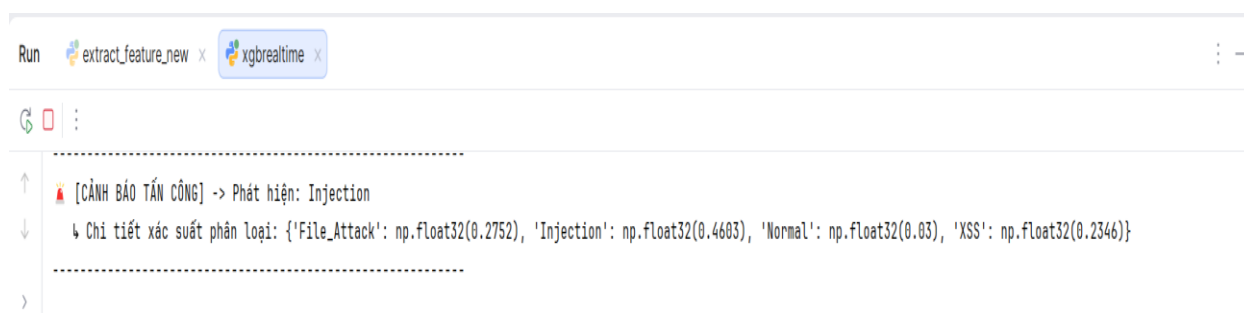
☐ Remember me

[Sign In](#)

[Forgot your password?](#)
[← Back to Home](#)

Hình 3.9 Thao tác gửi payload SQL injection vào form đăng nhập

- + Hệ thống phát hiện: Quá trình tiền xử lý ghi nhận sự gia tăng đột biến của các ký tự đặc biệt, đồng thời cờ `has_sql_keyword` được kích hoạt. Bộ đặc trưng này cung cấp đủ cơ sở để mô hình XGBoost nhận diện đây là hành vi can thiệp cơ sở dữ liệu và lập tức gán nhãn cảnh báo Injection



Hình 3.10 Terminal Python phân loại và cảnh báo nhãn Injection

- Kịch bản 4: Tấn công XSS
 - + Hành vi tấn công: Kẻ tấn công cố tình truyền các đoạn mã JavaScript độc hại vào phần tham số tìm kiếm hoặc thân gói tin (body) nhằm kiểm tra khả năng thực thi mã.

Search

```
<script> history.replaceState(null, null, '././../login'); document.body.innerHTML = "<br><br><br>"
```

Search

Index of /api/search?q=%3Cscript%3E%20history.replaceState(null,

Name	Size	Permissions
config/	4096	drwxr-xr-x
backup/	4096	drwxr-xr-x
logs/	4096	drwxrwxr-x
data/	4096	drwxr-xr-x
settings.conf	1047	-rw-r--r--
database.sql	89168	-rw-r--r--
.htaccess	892	-rw-r--r--
README.md	1929	-rw-r--r--

Hình 3.11 Thao tác gửi request chứa mã độc JavaScript (XSS) vào ô tìm kiếm

Krawl Login

Please log in to continue

Username

Password

☐ Remember me

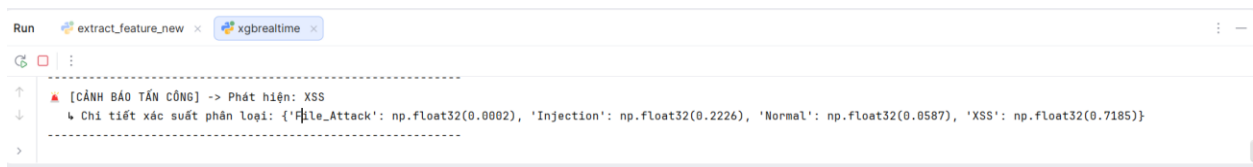
Sign In

Forgot your password?

[← Back to Home](#)

Hình 3.12 Thao tác gửi request chứa mã độc XSS vào form đăng nhập

- + Hệ thống phát hiện: Thuật toán đếm dấu ngoặc mở rộng ghi nhận giá trị `html_bracket_count` tăng cao, kết hợp với việc bắt trùng từ khóa trong danh sách `XSS_KEYWORDS`. Dữ liệu lập tức được mô hình XGBoost tổng hợp và phân loại thành công luồng truy cập với nhãn XSS



Hình 3.13 Terminal Python phân loại và cảnh báo nhận XSS

3.3. Tổng kết chương 3

Chương 3 đã trình bày chi tiết toàn bộ quá trình xây dựng môi trường thực nghiệm và kiểm thử hệ thống thông qua 4 kịch bản tương tác thực tế (Normal, Path Traversal, Injection, XSS). Dựa trên các kết quả trực quan thu được, nhóm nghiên cứu tiến hành tổng kết và đánh giá toàn diện về hệ thống như sau:

3.3.1. Các điểm mạnh đạt được

- Khả năng bẫy log chủ động và toàn vẹn: Hệ thống môi nhử Krawl Honeypot hoạt động ổn định, giả lập môi trường web tốt để "giữ chân" tin tặc và thu thập trọn vẹn các luồng dữ liệu tấn công gốc mà không gây rủi ro cho hệ thống thật.
- Hiệu năng xử lý thời gian thực: Mô đun tiền xử lý bằng ngôn ngữ Python thực hiện giải mã và bóc tách 18 đặc trưng cốt lõi từ gói tin HTTP cực kỳ nhanh chóng. Độ trễ thấp (15-25 ms) giúp hệ thống hoàn toàn đáp ứng được yêu cầu giám sát liên tục.
- Độ chính xác phân loại cao: Ứng dụng thành công thuật toán XGBoost vào bài toán phát hiện xâm nhập. Mô hình nhận diện và gán nhãn chính xác cả 4 kịch bản thực nghiệm mà không sinh ra cảnh báo giả làm nhiễu dữ liệu giám sát.

3.3.2. Những hạn chế tồn tại

- Quy mô tập dữ liệu kiểm thử: Quá trình thực nghiệm hiện tại chủ yếu dựa trên các kịch bản tương tác thủ công của nhóm. Quy mô dữ liệu chưa thực sự lớn và chưa bao phủ được toàn bộ các kỹ thuật lẩn tránh AI phức tạp của tin tặc.
- Mức độ tương tác của Honeypot: Krawl hiện tại tập trung vào việc đánh lừa ở tầng giao thức (luôn trả về HTTP 200 OK) nhưng chưa mô phỏng chi tiết được các logic nghiệp vụ phức tạp của một ứng dụng web thực tế (như cơ chế quản lý phiên đăng nhập đa lớp).
- Tính năng ngăn chặn tự động: Hệ thống mới chỉ dừng lại ở vai trò phát hiện, cảnh báo và lưu trữ log (tương tự IDS), chưa được tích hợp module để tự

động ngắt kết nối hoặc chặn dải IP độc hại ngay lập tức khi phát hiện tấn công.

3.3.3. Hướng phát triển tiếp theo

- Mở rộng nguồn thu thập dữ liệu: Triển khai hệ thống bẫy trên môi trường Internet thực tế để thu thập các luồng rà quét tự nhiên, giúp làm phong phú tập dữ liệu huấn luyện và nâng cao khả năng nhận diện các biến thể payload mới của mô hình XGBoost.
- Tích hợp cơ chế phản ứng chủ động (IPS/WAF): Liên kết kết quả phân loại của mô hình AI với tường lửa ứng dụng web để tự động cập nhật luật phòng ngự và chặn đứng các kết nối nguy hiểm theo thời gian thực.
- Trực quan hóa dữ liệu giám sát: Nghiên cứu tích hợp hệ thống với các nền tảng quản lý log tập trung để cung cấp giao diện Dashboard trực quan, giúp người quản trị dễ dàng theo dõi bản đồ tấn công và thống kê sự cố an toàn thông tin một cách chuyên nghiệp.

KẾT LUẬN

Trải qua quá trình nghiên cứu và thực nghiệm, đề tài xây dựng hệ thống Honeypot tích hợp Học máy nhằm thu thập và phân tích tấn công Web đã hoàn thành các mục tiêu cốt lõi đặt ra ban đầu. Mặc dù chỉ trong khuôn khổ của một môn học Chuyên đề cơ sở, hệ thống đã bước đầu chứng minh được tính khả thi và hiệu quả của phương pháp phòng thủ chủ động. Về mặt lý thuyết, nhóm thực hiện đã nắm vững nguyên lý hoạt động của hệ thống miễn nhiễm, hiểu rõ bản chất của các kỹ thuật khai thác lỗ hổng web phổ biến cũng như biết cách vận dụng nền tảng Học máy vào bài toán an toàn thông tin. Về mặt thực tiễn, đề tài đã thiết kế và triển khai thành công một đường ống dữ liệu khép kín trên môi trường Windows. Hệ thống đã tối ưu hóa khả năng thu thập dữ liệu bằng cách thiết lập máy chủ Nginx ở tuyến đầu để chủ động chặn bắt và trích xuất nguyên bản các yêu cầu truy cập, kết hợp với môi trường miễn nhiễm Krawl an toàn. Bên cạnh đó, việc xây dựng thành công các kịch bản tiền xử lý dữ liệu bằng Python và tích hợp mô hình Học máy để tự động phân loại, gom cụm các mã độc hại đã giúp giảm tải đáng kể thời gian phân tích thủ công.

Bên cạnh những kết quả tích cực đã đạt được, do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, đề tài vẫn còn tồn tại một số hạn chế nhất định. Hệ thống hiện tại chủ yếu được đánh giá dựa trên các kịch bản tấn công giả lập hoặc rà quét tự động trong thời gian ngắn, chưa có điều kiện cọ xát với các luồng dữ liệu lớn và phức tạp từ các cuộc tấn công có chủ đích trong thực tế. Hơn nữa, các thuật toán Học máy cơ bản được sử dụng tuy nhẹ và hoạt động nhanh nhưng đôi khi vẫn gặp khó khăn trong việc nhận diện các đoạn mã tấn công được làm rối tinh vi, dẫn đến một tỷ lệ nhỏ nhận diện sai. Ngoài ra, nền tảng Krawl chủ yếu tập trung giả lập ở tầng ứng dụng, do đó chưa thể ghi nhận sâu các hành vi leo thang đặc quyền ở cấp độ hệ điều hành.

Từ những hạn chế trên, đề tài mở ra nhiều hướng phát triển tiềm năng để có thể ứng dụng vào các hệ thống giám sát an toàn thông tin thực tế. Trong tương lai, hệ thống có thể nâng cấp mô hình phân tích bằng cách áp dụng các thuật toán Học sâu để xử lý tốt hơn ngữ cảnh của các chuỗi mã độc dài và phát hiện hiệu quả các lỗ hổng chưa từng được biết đến. Việc kết nối đầu ra của mô hình với các nền tảng giám sát tập trung để xây dựng các bảng điều khiển trực quan, tự động sinh cảnh báo cho quản trị viên ngay khi phát hiện hành vi bất thường cũng là một định hướng quan trọng. Cuối cùng, hệ thống có thể mở rộng mạng lưới miễn nhiễm bằng cách triển khai thêm các thành phần tương tác cao và phân tán trên nhiều môi trường đám mây khác nhau, qua đó thu thập nguồn dữ liệu tình báo mối đe dọa đa dạng và toàn diện hơn.

TÀI LIỆU THAM KHẢO

- [1] C. K. Ng, L. Pan, và Y. Xiang, *Honeypot Frameworks and Their Applications: A New Framework*. Springer, 2018.
- [2] J. Buzzio-Garcia, "Creation of a High-Interaction Honeypot System based-on Docker containers," trong *2021 Fifth World Conference on Smart Trends in Systems Security and Sustainability (WorldS4)*, 2021
- [3] U. Raut, M. Mokashi, A. Nagarkar, R. Sharma, và C. Talnikar, "Engaging Attackers with a Highly Interactive Honeypot System Using ChatGPT," trong *IEEE Conference Publication*, 2023.
- [4] K. Jiang, H. Zheng, "Design and Implementation of A Machine Learning Enhanced Web Honeypot System," trong *2020 13th International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI)*, 2020.
- [5] Vitaly V. Polyakov, Sergei A. Lapin, "Architecture of the Honeypot System for Studying Targeted Attacks," trong *2018 XIV International Scientific-Technical Conference on Actual Problems of Electronics Instrument Engineering (APEIE)*, 2018.
- [6] BlessedRebuS, "Krawl," GitHub, 2024. [Trực tuyến].
<https://github.com/BlessedRebuS/Krawl>. (Ngày truy cập: 10/05/2026).
- [7] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [8] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [9] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

PHỤ LỤC

Hướng dẫn triển khai mô hình

Bước 1: Tải mã nguồn về máy

Mở Command Prompt (CMD) hoặc PowerShell và tiến hành tải mã nguồn từ kho lưu trữ GitHub về thư mục làm việc:

```
git clone https://github.com/lhuy05/Honypot-ML.git
cd Honypot-ML
```

Bước 2: Khởi tạo hệ thống thu thập (Krawl-main) bằng Docker

Di chuyển vào thư mục krawl-main để khởi chạy các dịch vụ nền tảng đã được cấu hình sẵn trong Docker:

```
cd krawl-main
docker-compose up -d
```

Bước 3: Khởi chạy Web Server (Nginx)

Mở một cửa sổ CMD mới (hoặc quay lại thư mục gốc), di chuyển vào thư mục chứa Nginx và khởi chạy file thực thi:

```
cd Nginx\nginx-1.31.1
start nginx.exe
```

Bước 4: Cài đặt các thư viện Machine Learning phụ thuộc

Hệ thống yêu cầu các thư viện toán học và học máy lỗi để tính toán. Mở một cửa sổ CMD mới tại thư mục chứa mã nguồn Python (Honeypot-ML) và chạy lệnh:

```
cd ML
pip install -r requirements.txt
```

(Các thư viện cốt lõi bao gồm: pandas, numpy, xgboost, joblib).

Bước 5: Vận hành luồng phân tích thời gian thực (Real-time Pipeline)

Quá trình phân tích AI yêu cầu chạy song song hai tiến trình. Đảm bảo bạn đang đứng ở thư mục chứa hai tệp Python (extract_feature_new.py và xgbrealttime.py). Cần mở hai cửa sổ CMD riêng biệt:

- Tại CMD 1 (Tiến trình trích xuất đặc trưng):
 - + Tiến trình này sẽ liên tục đọc luồng dữ liệu từ tệp access.log của Nginx, bóc tách và tạo ra tệp trung gian realtime_features_outputnew1.csv.

```
python extract_feature_new.py
```

- Tại CMD 2 (Tiến trình dự đoán và Cảnh báo):
 - + Tiến trình này giám sát tệp CSV vừa được tạo, nạp dữ liệu vào mô hình XGBoost và hiển thị cảnh báo trực tiếp lên màn hình.

```
python xgbrealtime.py
```

Bước 5: Kiểm tra hoạt động của hệ thống

Bất kỳ truy cập mạng nào (bình thường hoặc chứa mã độc) đi qua Nginx sẽ ngay lập tức được bóc tách và hiển thị thông số trên CMD 1.

Gần như đồng thời, CMD 2 sẽ bắt được vector đặc trưng này và in ra màn hình kết quả phân loại (Ví dụ: [CẢNH BÁO TẤN CÔNG] -> Phát hiện: Injection). Để dừng hệ thống hoặc dừng Nginx, sử dụng Ctrl + C hoặc lệnh nginx -s quit.